



Complete Component Breakdown: SAT Compliance Platform

Yes, I understand perfectly. You want a **comprehensive technical inventory** of every single component that needs to be built, with complete detail on what each component does, how it works, and what it contains.

Let me break down the entire system into discrete, buildable components.



COMPONENT CATALOG STRUCTURE

I'll organize this as:

1. **Component Name**
 2. **Purpose** - What it does
 3. **Technical Details** - How it works
 4. **Inputs/Outputs** - Data flow
 5. **Dependencies** - What it needs
 6. **Files/Modules** - What you're actually building
-



AUTHENTICATION & AUTHORIZATION LAYER

Component 1: User Authentication Service

Purpose: Handle user registration, login, session management, and password operations.

Technical Details:

- JWT-based authentication with refresh tokens
- Email verification flow
- Password hashing with bcrypt (12 rounds)
- Session storage in Redis
- Rate limiting on auth endpoints

Inputs:

- Email, password (registration/login)
- Verification tokens
- Password reset tokens

Outputs:

- Access token (JWT, 15min expiry)
- Refresh token (30 day expiry)
- User session object

Dependencies:

- PostgreSQL (users table)
- Redis (session storage)
- Email service (verification emails)
- bcrypt library
- jsonwebtoken library

Files/Modules:

```
src/server/auth/
├── service.ts
│   ├── register(email, password, fullName)
│   ├── login(email, password)
│   ├── verifyEmail(token)
│   ├── requestPasswordReset(email)
│   ├── resetPassword(token, newPassword)
│   ├── refreshToken(refreshToken)
│   └── logout(userId)
├── middleware.ts
│   ├── requireAuth()
│   ├── requireRole(roles)
│   └── validateSession()
├── tokens.ts
│   ├── generateAccessToken(user)
│   ├── generateRefreshToken(user)
│   ├── verifyToken(token)
│   └── revokeToken(token)
└── validation.ts
    ├── validateEmail(email)
    ├── validatePassword(password)
    └── sanitizeInput(data)
```

...

Component 2: Role-Based Access Control (RBAC)

****Purpose:**** Manage permissions and access control across the application based on user roles.

****Technical Details:****

- Hierarchical role system: Owner > Admin > Accountant > User
- Permission-based actions (create, read, update, delete)
- Resource-level permissions (own vs. organization)
- Cached permission checks in Redis

****Inputs:****

- User ID
- Resource type (invoice, customer, etc.)
- Action type (create, read, update, delete)

****Outputs:****

- Boolean (allowed/denied)
- Available actions for resource
- Role capabilities object

****Dependencies:****

- PostgreSQL (users, organizations, roles)
- Redis (permission cache)
- Authentication service

****Files/Modules:****

...

src/server/rbac/

```
|— service.ts
|   |— checkPermission(userId, resource, action)
|   |— getUserPermissions(userId)
|   |— assignRole(userId, organizationId, role)
|   |— revokeRole(userId, organizationId)
|   |— getAvailableActions(userId, resourceType)
|— roles.ts
|   |— OWNER_PERMISSIONS
|   |— ADMIN_PERMISSIONS
|   |— ACCOUNTANT_PERMISSIONS
|   |— USER_PERMISSIONS
```

```

|— permissions.ts
|   |— INVOICE_PERMISSIONS
|   |— CUSTOMER_PERMISSIONS
|   |— EXPENSE_PERMISSIONS
|   |— REPORT_PERMISSIONS
|— middleware.ts
|   |— requirePermission(resource, action)
|   |— filterByPermissions(data, userId)
...

```

Component 3: Multi-Tenant Context Manager

****Purpose:**** Isolate data between organizations and manage tenant context throughout requests.

****Technical Details:****

- Middleware extracts organization from user context
- All queries automatically scoped to `organization_id`
- Tenant isolation at database level (Row-Level Security)
- Cross-tenant access prevention

****Inputs:****

- User session
- Request context

****Outputs:****

- Organization context object
- Scoped database queries

****Dependencies:****

- Authentication service
- PostgreSQL (RLS policies)
- Request middleware

****Files/Modules:****

...

src/server/multi-tenant/

```

|— middleware.ts
|   |— extractTenantContext()
|   |— validateTenantAccess()
|   |— injectTenantScope()
|— context.ts

```

```

|   |— TenantContext class
|   |— getCurrentOrganization()
|   |— switchOrganization(orgId)
|— database.ts
|   |— createScopedClient(organizationId)
|   |— enableRLS()
|   |— createRLSPolicies()
|— isolation.ts
|   |— validateCrossOrgAccess()
|   |— preventDataLeakage()
...

```

👤 ORGANIZATION MANAGEMENT

Component 4: Organization Service

****Purpose:**** Manage organization creation, configuration, and settings.

****Technical Details:****

- Organization profile management
- RFC validation against SAT format
- CFDI certificate storage (encrypted)
- PAC provider configuration
- Subscription/plan management

****Inputs:****

- Organization details (name, RFC, legal name, address)
- CFDI certificates (.cer, .key files)
- PAC credentials
- Subscription plan

****Outputs:****

- Organization object
- Certificate validation status
- Configuration status

****Dependencies:****

- PostgreSQL (organizations table)
- File storage (S3/R2 for certificates)
- Encryption service
- RFC validation service

****Files/Modules:****

...

src/server/organizations/

- service.ts
 - createOrganization(data)
 - updateOrganization(orgId, data)
 - getOrganization(orgId)
 - deleteOrganization(orgId)
 - uploadCertificates(orgId, cerFile, keyFile, password)
 - validateCertificates(cerFile, keyFile, password)
 - configurePAC(orgId, provider, credentials)
 - updateSettings(orgId, settings)
- validation.ts
 - validateRFC(rfc)
 - validateAddress(address)
 - validateTaxRegime(regime)
 - validateCertificateFiles(cer, key)
- certificates.ts
 - parseCertificate(cerFile)
 - extractCertificateInfo(cer)
 - validateCertificateExpiry(cer)
 - encryptPrivateKey(keyFile, password)
 - decryptPrivateKey(encrypted, password)
- types.ts
 - Organization interface
 - Address interface
 - CertificateInfo interface
 - PACConfig interface

...

Component 5: Team Management Service

****Purpose:**** Manage users within an organization, invitations, and role assignments.

****Technical Details:****

- User invitation system
- Email-based invitations with expiring tokens
- Role assignment and revocation
- User activity tracking

****Inputs:****

- Invitation data (email, role)

- User ID and organization ID
- Role changes

****Outputs:****

- Invitation status
- Team member list
- User roles and permissions

****Dependencies:****

- Organization service
- RBAC service
- Email service
- PostgreSQL (users, organizations)

****Files/Modules:****

...

src/server/team/

```

├── service.ts
│   ├── inviteUser(orgId, email, role, invitedBy)
│   ├── acceptInvitation(token, userData)
│   ├── cancelInvitation(invitationId)
│   ├── removeUser(orgId, userId)
│   ├── updateUserRole(orgId, userId, newRole)
│   ├── getTeamMembers(orgId)
│   └── transferOwnership(orgId, newOwnerId)
├── invitations.ts
│   ├── createInvitation(email, orgId, role)
│   ├── generateInvitationToken()
│   ├── validateInvitationToken(token)
│   └── getInvitation(token)
└── notifications.ts
    ├── sendInvitationEmail(email, token, orgName)
    └── sendRoleChangeNotification(user, newRole)

```

...

CUSTOMER MANAGEMENT

Component 6: Customer Service

****Purpose:**** Manage customer (receptor) information for invoicing.

****Technical Details:****

- CRUD operations for customers
- RFC validation and formatting
- SAT tax regime validation
- CFDI use code management
- Customer search and filtering

****Inputs:****

- Customer data (RFC, legal name, email, address, tax regime)
- Search queries
- Filter parameters

****Outputs:****

- Customer object
- Customer list (paginated)
- Validation results

****Dependencies:****

- PostgreSQL (customers table)
- RFC validation service
- SAT catalog service (tax regimes, CFDI uses)

****Files/Modules:****

...

src/server/customers/

```

├── service.ts
│   ├── createCustomer(orgId, data)
│   ├── updateCustomer(customerId, data)
│   ├── getCustomer(customerId)
│   ├── deleteCustomer(customerId)
│   ├── listCustomers(orgId, filters, pagination)
│   ├── searchCustomers(orgId, query)
│   ├── validateCustomer(data)
│   └── getCustomerInvoices(customerId)
├── validation.ts
│   ├── validateRFC(rfc)
│   ├── validateTaxRegime(regime)
│   ├── validateCFDIUse(use, taxRegime)
│   ├── validateEmail(email)
│   └── validateAddress(address)
└── repository.ts
    ├── findById(id)
    ├── findByRFC(orgId, rfc)
    ├── findByOrganization(orgId, options)
    └── create(customer)

```



```

|   |— update(id, data)
|   |— softDelete(id)
|— types.ts
|   |— Customer interface
|   |— CustomerFilters interface
|   |— CustomerAddress interface
...

```

Component 7: RFC Validation & Lookup Service

****Purpose:**** Validate RFC format and optionally lookup RFC details from SAT.

****Technical Details:****

- RFC format validation (physical vs. moral persons)
- Check digit validation
- Optional SAT API integration for RFC verification
- Cache validated RFCs

****Inputs:****

- RFC string
- Validation level (format only vs. SAT lookup)

****Outputs:****

- Validation result (valid/invalid)
- RFC type (physical/moral person)
- SAT registered information (if lookup enabled)

****Dependencies:****

- SAT API (optional)
- Redis (RFC cache)

****Files/Modules:****

...

src/server/rfc/

```

|— service.ts
|   |— validateRFC(rfc)
|   |— formatRFC(rfc)
|   |— getRFCType(rfc)
|   |— lookupRFC(rfc) // SAT API
|   |— validateCheckDigit(rfc)
|— validation.ts
|   |— validatePhysicalPersonRFC(rfc)

```

```

|   |— validateMoralPersonRFC(rfc)
|   |— validateGenericRFC(rfc)
|   |— calculateCheckDigit(rfc)
|— sat-api.ts
|   |— queryRFCStatus(rfc)
|   |— getRFCDetails(rfc)
|— constants.ts
|   |— RFC_PATTERNS
|   |— FORBIDDEN_WORDS
|   |— GENERIC_RFCS
...

```

📦 PRODUCT & SERVICE CATALOG

Component 8: Product/Service Management Service

****Purpose:**** Manage products and services catalog with pricing and SAT codes.

****Technical Details:****

- CRUD operations for products/services
- SKU management
- Pricing and tax configuration
- SAT product code assignment
- Unit of measure management
- Optional inventory tracking

****Inputs:****

- Product data (name, description, price, SAT code, tax rate)
- Search/filter parameters
- Inventory updates

****Outputs:****

- Product object
- Product list (paginated)
- Inventory status
- Pricing information

****Dependencies:****

- PostgreSQL (products table)
- SAT catalog service (product codes, units)
- AI service (SAT code suggestions)

****Files/Modules:****

...

src/server/products/

- service.ts
 - createProduct(orgId, data)
 - updateProduct(productId, data)
 - getProduct(productId)
 - deleteProduct(productId)
 - listProducts(orgId, filters, pagination)
 - searchProducts(orgId, query)
 - updateInventory(productId, quantity, operation)
 - checkStock(productId, quantity)
 - suggestSATCode(description)
- validation.ts
 - validateProduct(data)
 - validatePrice(price)
 - validateSATCode(code)
 - validateUnitOfMeasure(unit)
 - validateTaxRate(rate)
- inventory.ts
 - adjustStock(productId, quantity, reason)
 - getStockLevel(productId)
 - checkLowStock(orgId)
 - getInventoryHistory(productId)
- repository.ts
 - findById(id)
 - findBySKU(orgId, sku)
 - findByOrganization(orgId, options)
 - create(product)
 - update(id, data)
 - softDelete(id)
- types.ts
 - Product interface
 - ProductFilters interface
 - InventoryUpdate interface

...

🧠 AI SERVICES

Component 9: SAT Code Search Service (AI-Powered)

****Purpose:**** Provide intelligent search and suggestions for SAT product/service codes using embeddings and vector similarity.

****Technical Details:****

- Sentence transformer embeddings (multilingual model)
- Vector similarity search using pgvector
- Support for Spanish and English queries
- Fuzzy matching for typos
- Context-aware suggestions
- 55,000+ SAT codes indexed

****Inputs:****

- Search query (Spanish or English)
- Number of results to return
- Minimum similarity threshold

****Outputs:****

- Ranked list of SAT codes with scores
- Code, description, category
- Similarity score (0-1)

****Dependencies:****

- Python FastAPI service
- PostgreSQL with pgvector extension
- sentence-transformers library
- SAT catalog data

****Files/Modules:****

...

```
ai-service/app/routers/sat_search.py
├── search_sat_code(query, top_k, threshold)
├── search_by_category(category, query)
└── get_code_details(code)
```

```
ai-service/app/services/embedding.py
├── EmbeddingService class
├── generate_embedding(text)
├── generate_batch_embeddings(texts)
└── get_model()
```

```
ai-service/app/services/vector_search.py
├── VectorSearchService class
├── similarity_search(embedding, top_k, threshold)
└── search_with_filters(embedding, filters)
```

└─ get_similar_codes(code, top_k)

ai-service/app/models/sat_code.py

└─ SATCode model
└─ SATCodeResponse model
└─ SearchRequest model

ai-service/scripts/

└─ load_sat_catalog.py
└─ download_sat_catalog()
└─ parse_csv()
└─ insert_into_db()
└─ generate_embeddings.py
└─ load_existing_codes()
└─ generate_all_embeddings()
└─ update_database()

...

Component 10: Receipt OCR Service

****Purpose:**** Extract text and data from receipt images and PDFs using OCR.

****Technical Details:****

- Image preprocessing (deskewing, noise reduction)
- OCR using Tesseract or cloud service (Google Vision, AWS Textract)
- Text extraction and parsing
- Structured data extraction (amount, date, vendor, RFC)
- Support for multiple image formats

****Inputs:****

- Image file (JPEG, PNG, PDF)
- Language hint (Spanish)

****Outputs:****

- Extracted text
- Structured data (amount, date, vendor, RFC if present)
- Confidence scores

****Dependencies:****

- Tesseract OCR or cloud OCR service
- PIL/Pillow for image processing
- PostgreSQL (for caching results)

****Files/Modules:****

...

ai-service/app/routers/ocr.py

- |— extract_receipt(file)
- |— extract_invoice_xml(file)
- |— validate_extraction(data)

ai-service/app/services/ocr.py

- |— OCRService class
- |— extract_text(image)
- |— preprocess_image(image)
- |— parse_receipt_data(text)
- |— extract_rfc(text)
- |— extract_amount(text)
- |— extract_date(text)
- |— extract_vendor(text)

ai-service/app/services/image_processing.py

- |— resize_image(image, max_size)
- |— deskew_image(image)
- |— remove_noise(image)
- |— enhance_contrast(image)
- |— convert_to_grayscale(image)

ai-service/app/models/receipt.py

- |— ReceiptData model
- |— ExtractedField model
- |— OCRResult model

...

Component 11: Tax Assistant Chatbot (LLM-based)

****Purpose:**** Provide conversational AI assistance for tax questions and guidance using local LLM.

****Technical Details:****

- Local LLM (Llama 3.1 via Ollama)
- Context-aware responses
- Multi-turn conversations
- Spanish and English support
- Retrieval-augmented generation (RAG) with tax knowledge base

- Fallback to GPT-4 for complex queries

****Inputs:****

- User message
- Conversation history
- User context (organization, role)

****Outputs:****

- AI response
- Confidence score
- Source references (if RAG)

****Dependencies:****

- Ollama (local LLM runtime)
- Llama 3.1 model
- Vector database (for knowledge base)
- PostgreSQL (conversation history)

****Files/Modules:****

...

ai-service/app/routers/assistant.py

- └─ chat(message, conversation_id)
- └─ get_conversation_history(conversation_id)
- └─ clear_conversation(conversation_id)

ai-service/app/services/llm.py

- └─ LLMService class
- └─ generate_response(prompt, context)
- └─ stream_response(prompt, context)
- └─ fallback_to_gpt4(prompt)
- └─ get_model()

ai-service/app/services/rag.py

- └─ RAGService class
- └─ retrieve_relevant_docs(query, top_k)
- └─ build_context(query, docs)
- └─ augment_prompt(query, context)

ai-service/app/services/conversation.py

- └─ ConversationManager class
- └─ create_conversation(user_id)
- └─ add_message(conversation_id, message)
- └─ get_history(conversation_id, limit)
- └─ summarize_conversation(conversation_id)

```
ai-service/app/knowledge/
├── tax_guide.md
├── cfdi_guide.md
├── sat_regulations.md
└── embeddings/ (vector store)
...
---
```

INVOICE MANAGEMENT

Component 12: Invoice Service (Core)

****Purpose:**** Manage the entire invoice lifecycle from draft to cancellation.

****Technical Details:****

- CRUD operations for invoices
- Draft management
- Status workflow (draft → pending → stamped → sent → paid)
- Invoice validation
- Related invoices (CFDI relacionados)
- Multi-currency support
- Discount and tax calculations

****Inputs:****

- Invoice data (customer, items, amounts)
- Invoice items
- Payment terms
- Related invoice UUIDs

****Outputs:****

- Invoice object
- Validation results
- Invoice calculations (subtotal, tax, total)

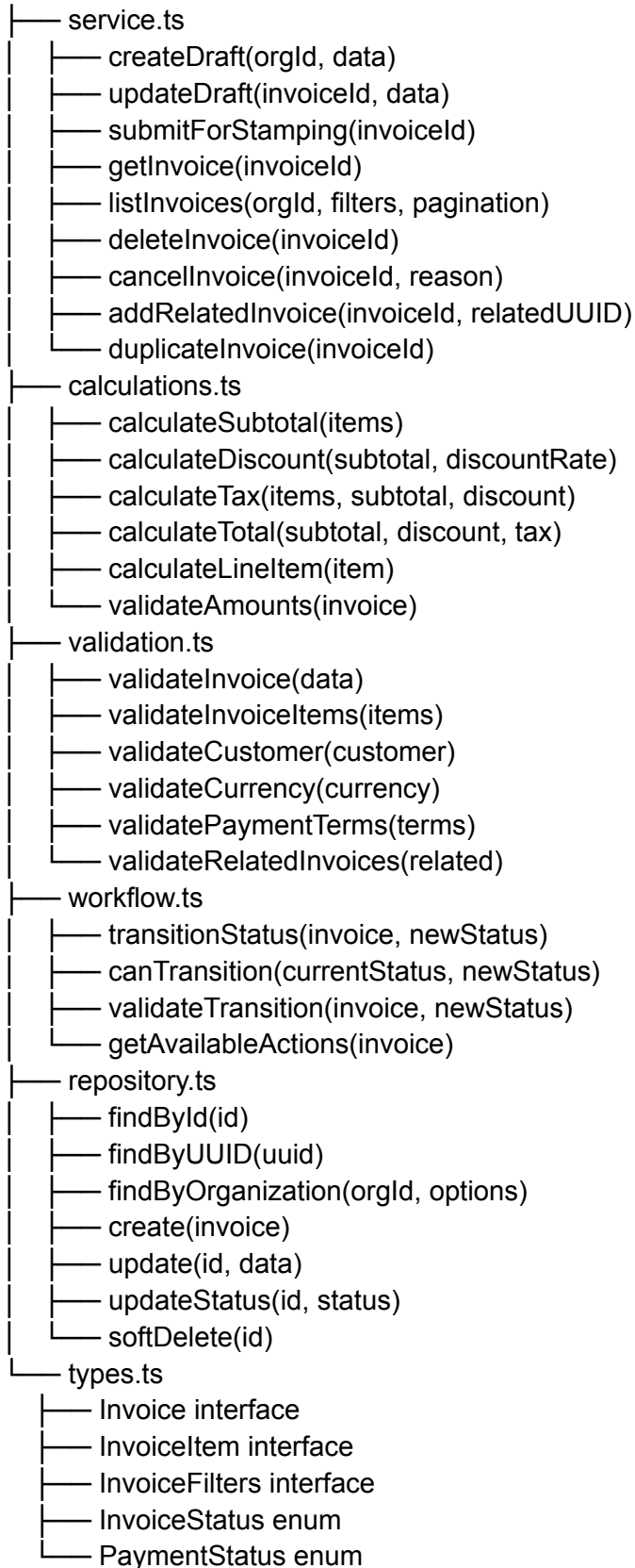
****Dependencies:****

- PostgreSQL (invoices, invoice_items tables)
- Customer service
- Product service
- Tax calculation service

****Files/Modules:****

...

src/server/invoices/



...

Component 13: CFDI XML Generator

****Purpose:**** Generate valid CFDI 4.0 XML documents according to SAT specifications.

****Technical Details:****

- CFDI 4.0 schema compliance
- XML structure generation
- Namespace handling
- Field formatting and validation
- Original string (cadena original) generation
- Support for complements (payment, foreign trade, etc.)

****Inputs:****

- Invoice object
- Organization fiscal data
- Customer fiscal data
- Invoice items

****Outputs:****

- CFDI XML string
- Original string for signing
- Validation report

****Dependencies:****

- xml2js or fast-xml-parser
- CFDI 4.0 schema (XSD)
- Organization service (certificates)
- SAT catalog service

****Files/Modules:****

...

packages/cfdi/src/

```
|— generator.ts
|   |— generateCFDI(invoice, org, customer)
|   |— buildComprobante()
|   |— buildEmisor(org)
|   |— buildReceptor(customer)
|   |— buildConceptos(items)
|   |— buildImpuestos(items)
|   |— buildComplemento()
```

```

├── formatXML(xmlObject)
├── schema.ts
│   ├── CFDI_NAMESPACE
│   ├── COMPROBANTE_ATTRIBUTES
│   ├── EMISOR_ATTRIBUTES
│   ├── RECEPTOR_ATTRIBUTES
│   └── CONCEPTO_ATTRIBUTES
├── validation.ts
│   ├── validateAgainstXSD(xml)
│   ├── validateStructure(xml)
│   ├── validateAmounts(xml)
│   ├── validateDates(xml)
│   └── validateCatalogs(xml)
├── cadena-original.ts
│   ├── generateCadenaOriginal(xml)
│   ├── applyXSLT(xml, xslt)
│   └── formatCadena(cadena)
├── complements/
│   ├── pagos.ts           // Complemento de pagos
│   ├── comercio-exterior.ts
│   └── nomina.ts
└── types.ts
    ├── CFDIComprobante interface
    ├── Emisor interface
    ├── Receptor interface
    ├── Concepto interface
    └── Impuestos interface

```

...

Component 14: Digital Signature Service (CSD)

****Purpose:**** Digitally sign CFDI XML documents using organization's CSD (Certificado de Sello Digital).

****Technical Details:****

- Load private key (.key file) with password
- Generate SHA-256 hash of original string
- RSA signature with private key
- Base64 encoding of signature
- Certificate validation

****Inputs:****

- Original string (cadena original)
- Organization CSD certificate and key
- Key password

****Outputs:****

- Digital signature (base64)
- Certificate number
- Seal (sello)

****Dependencies:****

- node-forge or crypto library
- Organization service (certificate retrieval)

****Files/Modules:****

...

packages/cfdi/src/

```

├── signer.ts
│   ├── signCFDI(cadenaOriginal, cert, key, password)
│   ├── loadPrivateKey(keyFile, password)
│   ├── loadCertificate(certFile)
│   ├── generateSignature(data, privateKey)
│   ├── base64Encode(signature)
│   └── getCertificateNumber(certificate)
├── crypto.ts
│   ├── decryptKey(encryptedKey, password)
│   ├── createHash(data, algorithm)
│   ├── sign(data, privateKey, algorithm)
│   └── verify(data, signature, publicKey)
└── certificate.ts
    ├── parseCertificate(certFile)
    ├── extractSerialNumber(cert)
    ├── extractPublicKey(cert)
    ├── validateCertificate(cert)
    ├── checkExpiration(cert)
    └── getCertificateInfo(cert)

```

...

Component 15: PAC Integration Service (Stamping)

****Purpose:**** Submit signed CFDI XML to PAC (Proveedor Autorizado de Certificación) for stamping.

****Technical Details:****

- Integration with multiple PACs (Finkok, SW, etc.)
- SOAP or REST API calls
- Handle stamping response
- Extract timbre fiscal electrónico (TFD)
- Retry logic for failures
- Error handling and mapping

****Inputs:****

- Signed CFDI XML
- PAC credentials
- Organization RFC

****Outputs:****

- Stamped XML (with TFD complement)
- UUID
- Stamp date
- SAT certificate number
- Stamping status

****Dependencies:****

- PAC API credentials
- SOAP client or HTTP client
- XML parser
- Organization service

****Files/Modules:****

...

src/server/pac/

```
|— service.ts
|   |— stampCFDI(xml, orgId)
|   |— cancelCFDI(uuid, orgId, reason)
|   |— queryStatus(uuid, orgId)
|   |— getPACProvider(orgId)
|— providers/
|   |— finkok.ts
|       |— stamp(xml, credentials)
|       |— cancel(uuid, credentials, reason)
|       |— query(uuid, credentials)
|       |— parseResponse(response)
|   |— sw.ts
|       |— stamp(xml, credentials)
|       |— cancel(uuid, credentials, reason)
|       |— query(uuid, credentials)
```

```

├── base.ts
│   └── PACProvider interface
├── soap-client.ts
│   ├── createClient(wsdl)
│   ├── call(method, params)
│   └── parseSOAPResponse(response)
├── tfd-parser.ts
│   ├── extractTFD(xml)
│   ├── parseTimbre(tfd)
│   ├── getUUID(tfd)
│   ├── getSATCertNumber(tfd)
│   └── getStampDate(tfd)
├── error-handler.ts
│   ├── mapPACError(error)
│   ├── isRetryable(error)
│   └── handleStampingError(error)
└── types.ts
    ├── StampRequest interface
    ├── StampResponse interface
    ├── CancelRequest interface
    └── PACCredentials interface

```

...

Component 16: PDF Generator Service

****Purpose:**** Generate professional PDF invoices from CFDI data.

****Technical Details:****

- PDF generation from invoice data
- QR code generation (URL to XML verification)
- SAT-compliant layout
- Multi-page support
- Company branding (logo, colors)
- Bilingual support (Spanish/English)

****Inputs:****

- Invoice object
- Organization branding settings
- Language preference

****Outputs:****

- PDF file (buffer or stream)

- PDF URL (after upload to storage)

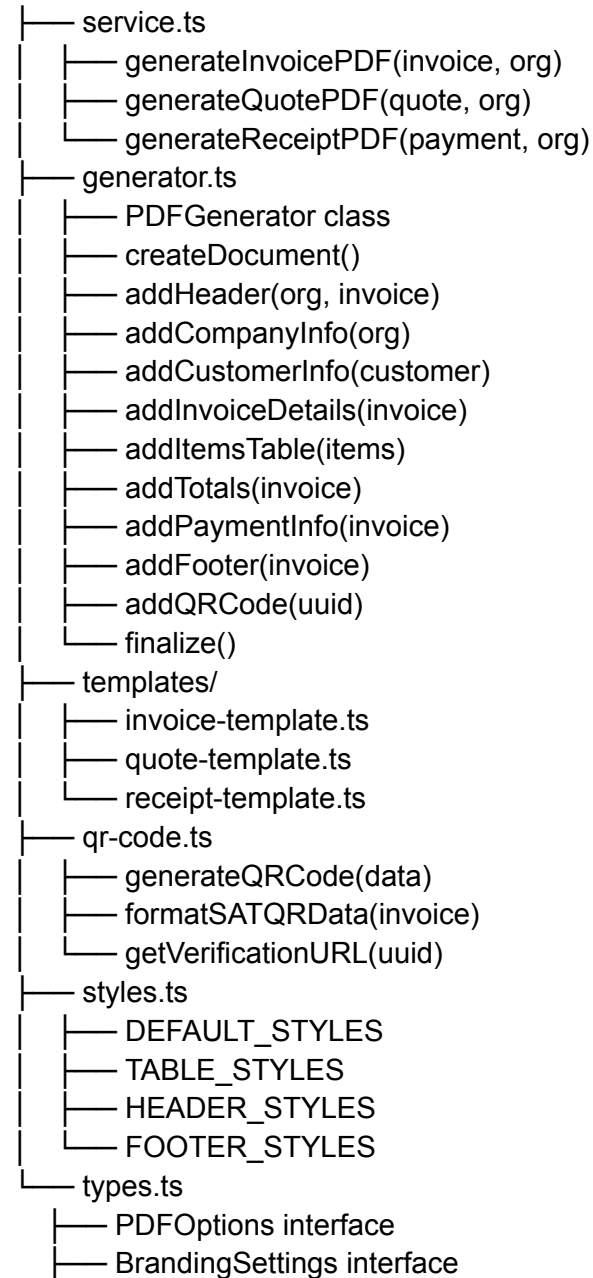
****Dependencies:****

- pdfkit or puppeteer
- qrcode library
- Image processing library
- Storage service

****Files/Modules:****

...

src/server/pdf/



└─ LayoutOptions interface
...

Component 17: Invoice Workflow Engine

Purpose: Manage invoice state transitions and automated actions.

Technical Details:

- State machine for invoice statuses
- Automated workflows (send email after stamping, etc.)
- Status transition validation
- Event triggers for status changes
- Scheduled actions (payment reminders)

Inputs:

- Invoice status change event
- Workflow configuration
- Trigger conditions

Outputs:

- Status transition result
- Triggered actions
- Workflow logs

Dependencies:

- Invoice service
- Notification service
- Job queue
- Event bus

Files/Modules:

...

src/server/workflows/

```
├─ engine.ts
│   ├── WorkflowEngine class
│   ├── executeWorkflow(invoice, event)
│   ├── validateTransition(from, to)
│   ├── triggerActions(invoice, status)
│   └─ scheduleFollowUp(invoice)
├─ state-machine.ts
│   ├── INVOICE_STATES
│   └─ VALID_TRANSITIONS
```



```

|   |— canTransition(from, to)
|   |— getNextStates(current)
|— actions/
|   |— send-email.ts
|   |— send-whatsapp.ts
|   |— update-accounting.ts
|   |— notify-team.ts
|   |— schedule-reminder.ts
|— triggers.ts
|   |— onStatusChange(invoice, oldStatus, newStatus)
|   |— onPaymentReceived(invoice, payment)
|   |— onDueDate(invoice)
|   |— onOverdue(invoice)
|— types.ts
|   |— WorkflowEvent interface
|   |— WorkflowAction interface
|   |— TransitionRule interface
...

```

💰 PAYMENT MANAGEMENT

Component 18: Payment Service

****Purpose:**** Record and manage payments against invoices.

****Technical Details:****

- Record payment transactions
- Link payments to invoices
- Support partial payments
- Update invoice payment status
- Generate complemento de pago (CFDI type P)
- Multi-currency payment handling

****Inputs:****

- Payment data (amount, date, method, reference)
- Invoice ID
- Payment method code

****Outputs:****

- Payment object
- Updated invoice status
- Complemento de pago XML (if PPD)

****Dependencies:****

- Invoice service
- CFDI generator (for payment complement)
- PAC service (for payment CFDI stamping)
- PostgreSQL (payments table)

****Files/Modules:****

...

src/server/payments/

```
├── service.ts
│   ├── recordPayment(invoiceId, data)
│   ├── updatePayment(paymentId, data)
│   ├── getPayment(paymentId)
│   ├── listPayments(filters, pagination)
│   ├── getInvoicePayments(invoiceId)
│   ├── calculateOutstanding(invoiceId)
│   ├── generatePaymentCFDI(paymentId)
│   └── voidPayment(paymentId)
├── calculations.ts
│   ├── calculatePaidAmount(invoiceId)
│   ├── calculateOutstanding(invoice, payments)
│   ├── determinePaymentStatus(invoice, payments)
│   ├── allocatePayment(payment, invoices)
│   └── handleExchangeRate(payment)
├── complement.ts
│   ├── generateComplementoPago(payment, invoice)
│   ├── buildDoctoRelacionado(invoice, payment)
│   ├── buildPago(payment)
│   └── validateComplement(xml)
├── repository.ts
│   ├── findById(id)
│   ├── findByInvoice(invoiceId)
│   ├── findByDateRange(orgId, start, end)
│   ├── create(payment)
│   ├── update(id, data)
│   └── softDelete(id)
└── types.ts
    ├── Payment interface
    ├── PaymentMethod enum
    ├── PaymentStatus enum
    └── ComplementoPago interface
```

...

Component 19: Payment Gateway Integration

****Purpose:**** Process online payments through Stripe or other payment providers.

****Technical Details:****

- Stripe API integration
- Payment intent creation
- Payment link generation
- Webhook handling for payment events
- Refund processing
- Payment method management

****Inputs:****

- Invoice data
- Payment amount
- Customer email
- Payment method

****Outputs:****

- Payment intent
- Payment link
- Payment status
- Transaction ID

****Dependencies:****

- Stripe API
- Webhook service
- Payment service
- Invoice service

****Files/Modules:****

...

src/server/payment-gateway/

```
|— service.ts
|   |— createPaymentIntent(invoice, amount)
|   |— createPaymentLink(invoice)
|   |— confirmPayment(paymentIntentId)
|   |— refundPayment(paymentId, amount)
|   |— getPaymentStatus(paymentIntentId)
|   |— savePaymentMethod(customerId, paymentMethodId)
|— stripe.ts
|   |— StripeService class
```

```

|   |— createCustomer(email, name)
|   |— createPaymentIntent(amount, currency, metadata)
|   |— createCheckoutSession(invoice)
|   |— constructWebhookEvent(body, signature)
|   |— handleRefund(chargeId, amount)
|— webhooks-handler.ts
|   |— handleWebhook(event)
|   |— onPaymentSucceeded(paymentIntent)
|   |— onPaymentFailed(paymentIntent)
|   |— onRefundCreated(refund)
|   |— onChargeDisputed(dispute)
|— payment-links.ts
|   |— generatePaymentLink(invoice, expiresAt)
|   |— getPaymentLinkStatus(linkId)
|   |— expirePaymentLink(linkId)
|— types.ts
|   |— PaymentIntent interface
|   |— PaymentLink interface
|   |— WebhookEvent interface
|   |— RefundRequest interface
...

```

📁 EXPENSE MANAGEMENT

Component 20: Expense Service

****Purpose:**** Track and manage business expenses for tax deductions.

****Technical Details:****

- Record expenses manually
- Upload receipts/XMLs
- CFDI validation for deductibility
- Expense categorization
- Approval workflows
- Expense reports

****Inputs:****

- Expense data (vendor, amount, date, category)
- Receipt image or CFDI XML
- Expense category

****Outputs:****

- Expense object
- Validation status
- Deductibility status
- Extracted data from receipt

****Dependencies:****

- PostgreSQL (expenses table)
- Storage service (receipts)
- OCR service (receipt scanning)
- CFDI validator

****Files/Modules:****

...

src/server/expenses/

```

├── service.ts
│   ├── createExpense(orgId, data)
│   ├── updateExpense(expenseId, data)
│   ├── getExpense(expenseId)
│   ├── listExpenses(orgId, filters, pagination)
│   ├── deleteExpense(expenseId)
│   ├── uploadReceipt(expenseId, file)
│   ├── validateCFDI(expenseId, xml)
│   ├── categorizeExpense(expenseId, category)
│   └── approveExpense(expenseId)
├── validation.ts
│   ├── validateExpense(data)
│   ├── validateCFDIForDeduction(xml)
│   ├── checkRFCMatch(xml, orgRFC)
│   ├── validateExpenseDate(date)
│   └── isDeductible(expense)
├── categorization.ts
│   ├── EXPENSE_CATEGORIES
│   ├── suggestCategory(description)
│   ├── getCategoryRules(category)
│   └── validateCategory(expense, category)
├── ocr-integration.ts
│   ├── extractExpenseData(receiptImage)
│   ├── parseReceiptText(text)
│   ├── autoFillExpense(ocrData)
│   └── validateExtraction(data)
└── reports.ts
    ├── generateExpenseReport(orgId, dateRange)
    ├── getExpensesByCategory(orgId, dateRange)
    └── getDeductibleExpenses(orgId, period)

```

```

|   └─ exportExpenses(orgId, filters, format)
|   └─ repository.ts
|       └─ findById(id)
|       └─ findByOrganization(orgId, options)
|       └─ findByDateRange(orgId, start, end)
|       └─ create(expense)
|       └─ update(id, data)
|       └─ softDelete(id)
|   └─ types.ts
|       └─ Expense interface
|       └─ ExpenseCategory enum
|       └─ ExpenseStatus enum
|       └─ ExpenseFilters interface
...

```

ACCOUNTING & BOOKKEEPING

Component 21: Chart of Accounts Service

****Purpose:**** Manage the organization's chart of accounts for double-entry bookkeeping.

****Technical Details:****

- Hierarchical account structure
- Account types (Asset, Liability, Equity, Revenue, Expense)
- Default chart of accounts template
- Custom account creation
- Account balance tracking
- Account history

****Inputs:****

- Account data (code, name, type, parent)
- Account code
- Balance updates

****Outputs:****

- Account object
- Account hierarchy
- Current balance
- Account list

****Dependencies:****

- PostgreSQL (chart_of_accounts table)

****Files/Modules:****

...

src/server/accounting/

- |— chart-of-accounts.ts
 - |— createAccount(orgId, data)
 - |— updateAccount(accountId, data)
 - |— getAccount(accountId)
 - |— listAccounts(orgId, filters)
 - |— deleteAccount(accountId)
 - |— getAccountHierarchy(orgId)
 - |— getAccountBalance(accountId, date)
 - |— createDefaultChart(orgId)
- |— account-types.ts
 - |— ASSET_ACCOUNTS
 - |— LIABILITY_ACCOUNTS
 - |— EQUITY_ACCOUNTS
 - |— REVENUE_ACCOUNTS
 - |— EXPENSE_ACCOUNTS
 - |— getAccountType(code)
- |— templates/
 - |— mexico-chart.ts // Mexican accounting standards
 - |— us-chart.ts // US GAAP
 - |— custom-chart.ts
- |— validation.ts
 - |— validateAccount(data)
 - |— validateAccountCode(code)
 - |— validateHierarchy(parent, child)
 - |— canDeleteAccount(accountId)
- |— types.ts
 - |— Account interface
 - |— AccountType enum
 - |— AccountHierarchy interface

...

Component 22: Journal Entry Service

****Purpose:**** Record and manage double-entry bookkeeping journal entries.

****Technical Details:****

- Create journal entries (manual and automatic)
- Debit/credit validation (must balance)

- Post journal entries
- Reverse entries
- Link to source documents (invoices, payments, expenses)
- Entry templates

****Inputs:****

- Journal entry data (date, description)
- Entry lines (account, debit, credit)
- Source document reference

****Outputs:****

- Journal entry object
- Posting status
- Validation results

****Dependencies:****

- Chart of accounts service
- PostgreSQL (journal_entries, journal_entry_lines tables)

****Files/Modules:****

...

src/server/accounting/

```

├── journal-entries.ts
│   ├── createEntry(orgId, data)
│   ├── updateEntry(entryId, data)
│   ├── getEntry(entryId)
│   ├── listEntries(orgId, filters, pagination)
│   ├── deleteEntry(entryId)
│   ├── postEntry(entryId)
│   ├── reverseEntry(entryId, date)
│   └── createFromTemplate(templateId, data)
├── validation.ts
│   ├── validateEntry(entry)
│   ├── validateBalance(lines)
│   ├── checkDebitsCreditsEqual(lines)
│   ├── validateAccounts(lines)
│   └── canPost(entry)
├── automatic-entries.ts
│   ├── createFromInvoice(invoice)
│   ├── createFromPayment(payment)
│   ├── createFromExpense(expense)
│   ├── createFromAdjustment(adjustment)
│   └── getEntryTemplate(type)
└── templates.ts

```



```

|   |— INVOICE_ENTRY_TEMPLATE
|   |— PAYMENT_ENTRY_TEMPLATE
|   |— EXPENSE_ENTRY_TEMPLATE
|   |— ADJUSTMENT_ENTRY_TEMPLATE
|   |
|   |— posting.ts
|   |   |— postEntries(entryIds)
|   |   |— unpostEntry(entryId)
|   |   |— updateAccountBalances(entry)
|   |   |— validatePostingDate(date)
|   |
|   |— types.ts
|       |— JournalEntry interface
|       |— JournalEntryLine interface
|       |— EntryTemplate interface
...

```

Component 23: Financial Reports Generator

****Purpose:**** Generate standard financial statements and reports.

****Technical Details:****

- Income statement (P&L)
- Balance sheet
- Cash flow statement
- Trial balance
- General ledger
- Custom date ranges
- Export to PDF/Excel

****Inputs:****

- Organization ID
- Report type
- Date range or period
- Comparison periods (optional)

****Outputs:****

- Report data structure
- Formatted report
- PDF or Excel file

****Dependencies:****

- Journal entry service
- Chart of accounts service

- PostgreSQL (accounting data)
- PDF generator

****Files/Modules:****

...

src/server/reports/

- financial-reports.ts
 - generateIncomeStatement(orgId, start, end)
 - generateBalanceSheet(orgId, asOfDate)
 - generateCashFlow(orgId, start, end)
 - generateTrialBalance(orgId, asOfDate)
 - generateGeneralLedger(orgId, start, end)
 - generateCustomReport(orgId, config)
- income-statement.ts
 - calculateRevenue(orgId, start, end)
 - calculateExpenses(orgId, start, end)
 - calculateGrossProfit(revenue, cogs)
 - calculateNetIncome(revenue, expenses)
 - formatIncomeStatement(data)
- balance-sheet.ts
 - calculateAssets(orgId, date)
 - calculateLiabilities(orgId, date)
 - calculateEquity(orgId, date)
 - validateBalance(assets, liabilities, equity)
 - formatBalanceSheet(data)
- cash-flow.ts
 - calculateOperatingActivities(orgId, start, end)
 - calculateInvestingActivities(orgId, start, end)
 - calculateFinancingActivities(orgId, start, end)
 - calculateNetCashFlow(activities)
 - formatCashFlow(data)
- trial-balance.ts
 - getAccountBalances(orgId, date)
 - calculateDebits(accounts)
 - calculateCredits(accounts)
 - formatTrialBalance(accounts)
- export.ts
 - exportToPDF(report, format)
 - exportToExcel(report)
 - exportToCSV(report)
 - emailReport(report, recipients)
- types.ts
 - IncomeStatement interface
 - BalanceSheet interface

```
├── CashFlow interface
├── ReportConfig interface
...
```

TAX CALCULATION & COMPLIANCE

Component 24: Tax Calculation Engine

****Purpose:**** Calculate Mexican taxes (IVA, ISR) for reporting periods.

****Technical Details:****

- Monthly/bimonthly IVA calculation
- IVA charged vs. IVA paid
- Provisional ISR calculation
- Tax withholdings
- Deduction validation
- Tax period management

****Inputs:****

- Organization ID
- Tax period (month, bimonth, quarter)
- Income and expense data

****Outputs:****

- IVA payable/refundable
- ISR payable
- Tax summary
- Calculation breakdown

****Dependencies:****

- Invoice service
- Expense service
- PostgreSQL (tax_periods table)

****Files/Modules:****

...

src/server/tax/

```
├── calculation-engine.ts
│   ├── calculateIVA(orgId, period)
│   ├── calculateISR(orgId, period)
│   ├── calculateWithholdings(orgId, period)
│   └── getTaxSummary(orgId, period)
```

```

├── saveTaxPeriod(orgId, period, calculations)
├── iva-calculator.ts
│   ├── calculateIVACHarged(invoices)
│   ├── calculateIVAPaid(expenses)
│   ├── calculateIVABalance(charged, paid)
│   ├── handleIVAWithholdings(invoices)
│   └── formatIVADeclaration(data)
├── isr-calculator.ts
│   ├── calculateGrossIncome(invoices)
│   ├── calculateDeductions(expenses)
│   ├── calculateNetIncome(gross, deductions)
│   ├── applyTaxRates(income)
│   ├── calculateProvisionalISR(income, regime)
│   └── formatISRDeclaration(data)
├── deductions.ts
│   ├── validateDeduction(expense)
│   ├── getDeductibleAmount(expense)
│   ├── applyDeductionLimits(expenses)
│   ├── categorizeDeductions(expenses)
│   └── getDeductionRules(category)
├── tax-regimes.ts
│   ├── REGIMEN_GENERAL
│   ├── RIF_REGIME
│   ├── RESICO_REGIME
│   ├── getTaxRates(regime, income)
│   └── getDeductionRules(regime)
├── periods.ts
│   ├── createTaxPeriod(orgId, type, year, period)
│   ├── getTaxPeriod(orgId, date)
│   ├── closeTaxPeriod(periodId)
│   ├── getNextDueDate(period)
│   └── isOverdue(period)
└── types.ts
    ├── TaxPeriod interface
    ├── IVACalculation interface
    ├── ISRCalculation interface
    └── TaxSummary interface

```

...

Component 25: Tax Filing Assistant

****Purpose:**** Guide users through tax declaration filing and track filing status.

****Technical Details:****

- Step-by-step filing wizard
- Pre-fill forms with calculated data
- Filing deadline tracking
- Filing status management
- Integration with SAT portal (future)
- Filing history

****Inputs:****

- Tax period
- Calculated tax amounts
- Organization fiscal data

****Outputs:****

- Pre-filled tax forms
- Filing checklist
- Filing status
- Submission confirmation

****Dependencies:****

- Tax calculation engine
- Organization service
- Notification service

****Files/Modules:****

...

src/server/tax/

```
|— filing-assistant.ts
|   |— startFilingWizard(orgId, period)
|   |— getFilingChecklist(orgId, period)
|   |— validateFilingData(data)
|   |— generateDeclarationForm(period)
|   |— submitFiling(periodId, data)
|   |— getFilingStatus(periodId)
|   |— downloadConfirmation(periodId)
|— wizard-steps.ts
|   |— Step1_ReviewIncome()
|   |— Step2_ReviewExpenses()
|   |— Step3_ReviewCalculations()
|   |— Step4_ConfirmSubmission()
|   |— Step5_Complete()
|— forms.ts
|   |— generateIVAForm(period)
```

```

|   |— generateISRForm(period)
|   |— preFillForm(form, period)
|   |— validateForm(form)
|— reminders.ts
|   |— scheduleFilingReminder(period)
|   |— sendDeadlineNotification(period)
|   |— sendOverdueAlert(period)
|   |— getUpcomingDeadlines(orgId)
|— types.ts
|   |— FilingWizardState interface
|   |— FilingChecklist interface
|   |— TaxForm interface
...

```

📱 WHATSAPP INTEGRATION

Component 26: WhatsApp Business API Integration

****Purpose:**** Integrate with WhatsApp Business API for messaging and bot functionality.

****Technical Details:****

- Meta WhatsApp Business API
- Webhook for incoming messages
- Message templates (for notifications)
- Media handling (images, PDFs)
- Session management
- Rate limiting

****Inputs:****

- Incoming webhook events
- Outgoing message data
- Phone numbers
- Message templates

****Outputs:****

- Message delivery status
- Incoming message objects
- Template message IDs

****Dependencies:****

- Meta Business account
- WhatsApp Business API credentials

- Webhook endpoint
- PostgreSQL (conversations, messages)

****Files/Modules:****

...

services/whatsapp/src/

- api-client.ts
 - WhatsAppAPIClient class
 - sendMessage(to, text)
 - sendTemplate(to, templateName, params)
 - sendMedia(to, mediaType, url)
 - markAsRead(messageId)
 - getMediaUrl(mediaId)
- webhook-handler.ts
 - handleWebhook(event)
 - verifyWebhook(token, challenge)
 - onMessageReceived(message)
 - onStatusUpdate(status)
 - onMediaReceived(media)
- conversation-manager.ts
 - getOrCreateConversation(phoneNumber, orgId)
 - addMessage(conversationId, message)
 - getConversationHistory(conversationId)
 - closeConversation(conversationId)
 - assignToUser(conversationId, userId)
- templates.ts
 - INVOICE_SENT_TEMPLATE
 - PAYMENT_REMINDER_TEMPLATE
 - PAYMENT_RECEIVED_TEMPLATE
 - OVERDUE_NOTICE_TEMPLATE
 - sendTemplateMessage(to, template, params)
- media-handler.ts
 - downloadMedia(mediaId)
 - uploadMedia(file)
 - getMediaUrl(mediaId)
 - deleteMedia(mediaId)
- types.ts
 - WhatsAppMessage interface
 - WebhookEvent interface
 - MessageTemplate interface
 - Conversation interface

...

Component 27: WhatsApp Chatbot (Intent Recognition)

****Purpose:**** Understand user intent from WhatsApp messages and route to appropriate handlers.

****Technical Details:****

- Natural language understanding (NLU)
- Intent classification (invoice_status, create_invoice, help, etc.)
- Entity extraction (invoice numbers, dates, amounts)
- Context management for multi-turn conversations
- Fallback to human support

****Inputs:****

- User message text
- Conversation context
- User/organization context

****Outputs:****

- Recognized intent
- Extracted entities
- Confidence score
- Suggested response or action

****Dependencies:****

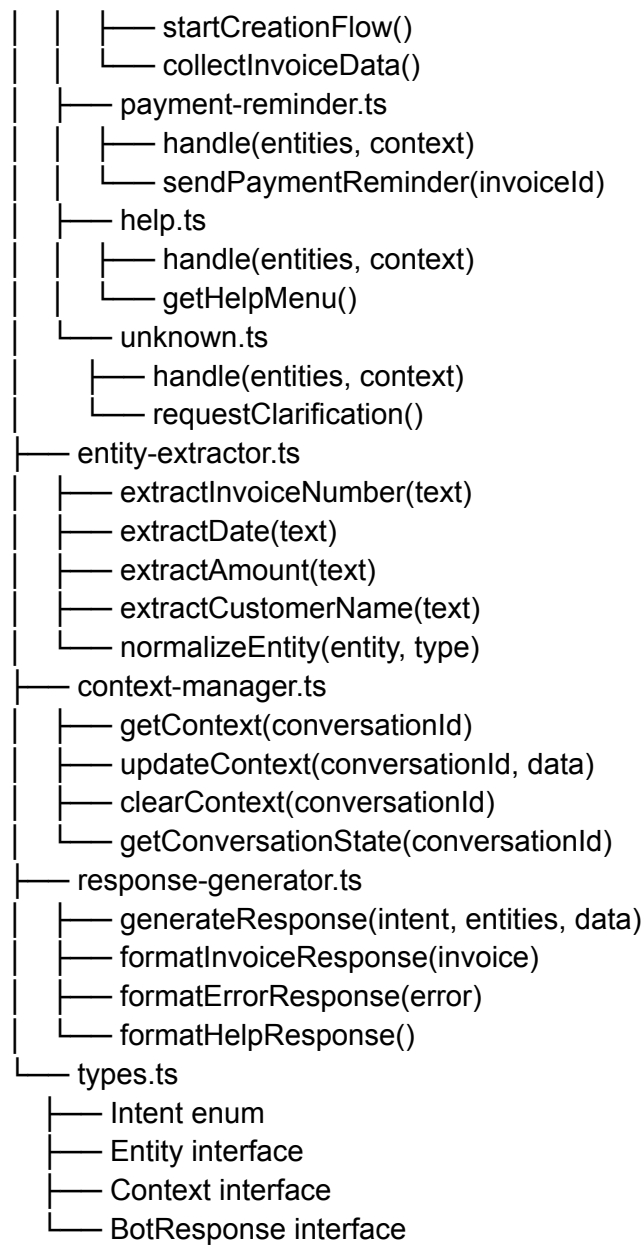
- AI service (local LLM or NLU model)
- Conversation manager
- Invoice service, customer service, etc.

****Files/Modules:****

...

services/whatsapp/src/bot/

```
|— intent-recognizer.ts
|   |— recognizeIntent(message, context)
|   |— classifyIntent(text)
|   |— extractEntities(text, intent)
|   |— getConfidence(prediction)
|   |— getFallbackIntent()
|— intents/
|   |— invoice-status.ts
|       |— handle(entities, context)
|       |— getInvoiceStatus(uuid)
|   |— create-invoice.ts
|       |— handle(entities, context)
```

— — —

Component 28: WhatsApp Notification Service

****Purpose:**** Send automated notifications via WhatsApp for key events.

****Technical Details:****

- Event-driven notification triggers
- Template message composition
- Notification preferences per user

- Batch notifications
- Delivery tracking

****Inputs:****

- Event type (invoice_sent, payment_received, etc.)
- Recipient phone number
- Event data (invoice, payment, etc.)

****Outputs:****

- Notification sent status
- Message ID
- Delivery status

****Dependencies:****

- WhatsApp API client
- Event bus or queue
- Notification preferences service

****Files/Modules:****

...

services/whatsapp/src/notifications/

```

|— service.ts
|   |— sendNotification(type, recipient, data)
|   |— sendBatchNotifications(notifications)
|   |— scheduleNotification(type, recipient, data, sendAt)
|   |— cancelNotification(notificationId)
|   |— getNotificationStatus(notificationId)
|— triggers.ts
|   |— onInvoiceCreated(invoice)
|   |— onInvoiceSent(invoice)
|   |— onPaymentReceived(payment)
|   |— onPaymentOverdue(invoice)
|   |— onInvoiceCancelled(invoice)
|   |— onExpenseApproved(expense)
|— templates/
|   |— invoice-created.ts
|   |— invoice-sent.ts
|   |— payment-received.ts
|   |— payment-reminder.ts
|   |— payment-overdue.ts
|   |— monthly-summary.ts
|— preferences.ts
|   |— getUserPreferences(userId)
|   |— updatePreferences(userId, preferences)

```

```

|   |— shouldSendNotification(userId, type)
|   |— getNotificationChannel(userId, type)
|— types.ts
|   |— NotificationType enum
|   |— NotificationPreferences interface
|   |— ScheduledNotification interface
...

```

COMMUNICATION SERVICES

Component 29: Email Service

****Purpose:**** Send transactional and notification emails.

****Technical Details:****

- SMTP integration or email service (SendGrid, Postmark)
- Email templates (HTML + plain text)
- Attachment support (PDF invoices, XML files)
- Bounce and spam handling
- Email tracking (opens, clicks)

****Inputs:****

- Recipient email
- Email type
- Template data
- Attachments (optional)

****Outputs:****

- Send status
- Email ID
- Delivery status

****Dependencies:****

- Email service provider (SendGrid, etc.)
- Template engine (Handlebars, React Email)
- Storage service (for attachments)

****Files/Modules:****

...

src/server/email/

```

|— service.ts
|   |— sendEmail(to, subject, template, data)

```

```

├── sendWithAttachments(to, subject, template, data, files)
├── sendBatch(emails)
├── getEmailStatus(emailId)
├── resendEmail(emailId)
├── templates/
│   ├── invoice-email.tsx
│   ├── payment-reminder.tsx
│   ├── welcome-email.tsx
│   ├── password-reset.tsx
│   ├── invitation-email.tsx
│   └── monthly-statement.tsx
├── providers/
│   ├── sendgrid.ts
│   │   ├── send(email)
│   │   ├── sendBatch(emails)
│   │   └── getStatus(emailId)
│   ├── postmark.ts
│   │   ├── send(email)
│   │   └── sendWithTemplate(template, data)
│   └── base.ts
│       └── EmailProvider interface
├── renderer.ts
│   ├── renderTemplate(template, data)
│   ├── renderReactEmail(component, props)
│   ├── generatePlainText(html)
│   └── inlineCSS(html)
├── attachments.ts
│   ├── attachInvoicePDF(invoice)
│   ├── attachInvoiceXML(invoice)
│   ├── prepareAttachment(file)
│   └── validateAttachment(file)
├── types.ts
│   ├── Email interface
│   ├── EmailTemplate interface
│   ├── Attachment interface
│   └── SendResult interface
...

```

📁 STORAGE & FILE MANAGEMENT

Component 30: File Storage Service

****Purpose:**** Manage file uploads and storage (invoices, receipts, certificates).

****Technical Details:****

- S3-compatible storage (AWS S3, Cloudflare R2, MinIO)
- File upload with validation
- Signed URLs for secure access
- File deletion and cleanup
- Virus scanning (optional)
- CDN integration

****Inputs:****

- File buffer or stream
- File metadata (type, organization, etc.)
- Access permissions

****Outputs:****

- File URL
- File ID
- Storage location

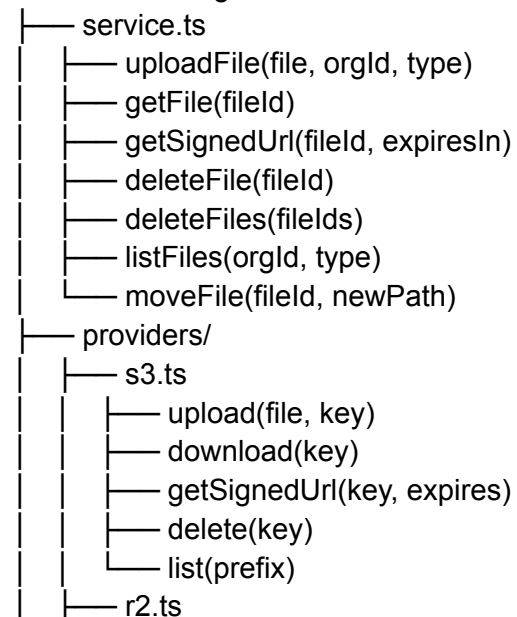
****Dependencies:****

- S3 SDK or storage provider API
- File validation library
- Virus scanner (ClamAV, etc.)

****Files/Modules:****

...

src/server/storage/



```

|   |   | upload(file, key)
|   |   | getPublicUrl(key)
|   |   |
|   |   | base.ts
|   |   |   | StorageProvider interface
|   |   |
|   | validation.ts
|   |   | validateFileType(file, allowedTypes)
|   |   | validateFileSize(file, maxSize)
|   |   | sanitizeFileName(filename)
|   |   | validateImage(file)
|   |   | validatePDF(file)
|   |
|   | virus-scan.ts
|   |   | scanFile(file)
|   |   | isInfected(scanResult)
|   |   | quarantineFile(fileId)
|   |
|   | paths.ts
|   |   | getFilePath(orgId, type, filename)
|   |   | getXMLPath(orgId, invoiceId)
|   |   | getPDFPath(orgId, invoiceId)
|   |   | getReceiptPath(orgId, expenseId)
|   |
|   | types.ts
|   |   | UploadResult interface
|   |   | FileMetadata interface
|   |   | FileType enum
|   |   | StorageConfig interface
|   |
|   ...

```

🛎 NOTIFICATION SYSTEM

Component 31: Notification Service

****Purpose:**** Centralized notification system for in-app, email, and WhatsApp notifications.

****Technical Details:****

- Multi-channel notification delivery
- Notification preferences management
- Notification history and read status
- Batch notifications
- Priority and urgency handling

****Inputs:****

- User ID or organization ID
- Notification type

- Notification data
- Delivery channels

****Outputs:****

- Notification ID
- Delivery status per channel
- Read status

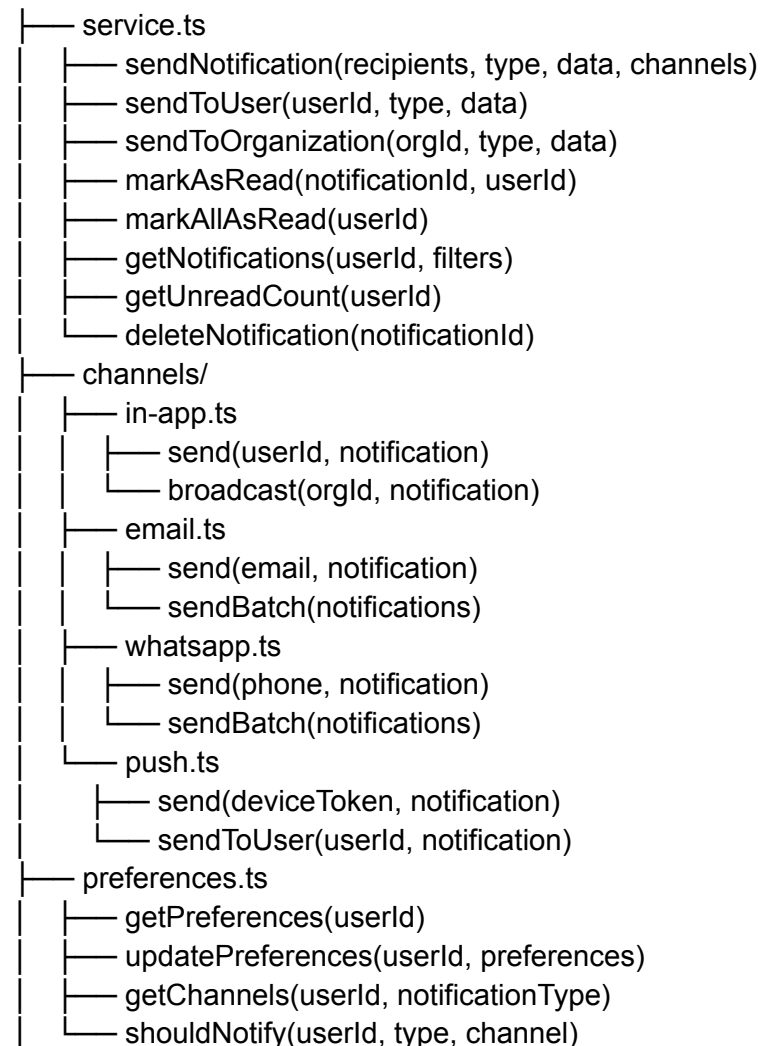
****Dependencies:****

- Email service
- WhatsApp service
- Push notification service (future)
- PostgreSQL (notifications table)

****Files/Modules:****

...

src/server/notifications/



```

|— types-config.ts
|   |— NOTIFICATION_TYPES
|   |— DEFAULT_PREFERENCES
|   |— NOTIFICATION_PRIORITIES
|   |— getNotificationConfig(type)
|— batching.ts
|   |— queueNotification(notification)
|   |— processBatch()
|   |— groupByChannel(notifications)
|   |— sendBatch(channel, notifications)
|— types.ts
|   |— Notification interface
|   |— NotificationChannel enum
|   |— NotificationPreferences interface
|   |— NotificationStatus interface
...

```

BACKGROUND JOBS & QUEUE

Component 32: Job Queue Service

****Purpose:**** Manage asynchronous background jobs with BullMQ and Redis.

****Technical Details:****

- Job queue with priority and delay
- Retry logic with exponential backoff
- Job scheduling (cron-like)
- Job monitoring and metrics
- Concurrent job processing
- Failed job handling

****Inputs:****

- Job type
- Job payload
- Priority and delay
- Retry configuration

****Outputs:****

- Job ID
- Job status
- Job result or error

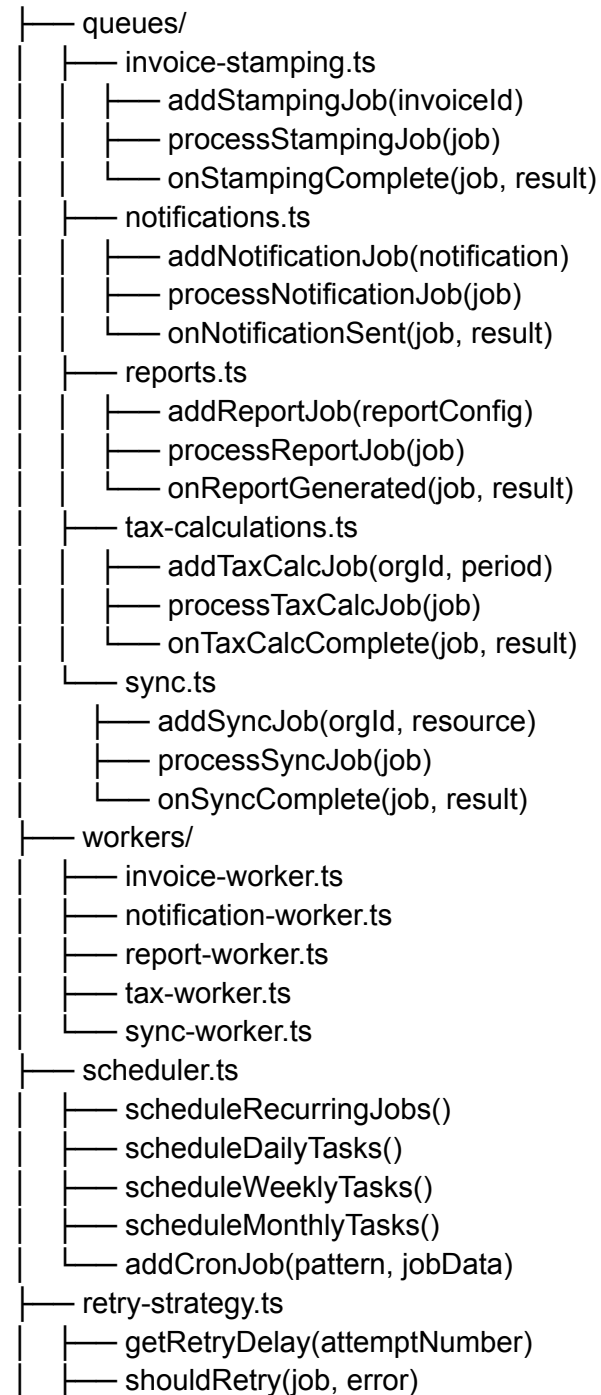
****Dependencies:****

- Redis
- BullMQ library
- Worker processes

****Files/Modules:****

...

services/workers/src/



```

|   |   |— getMaxAttempts(jobType)
|   |   |— handleFailedJob(job, error)
|   |— monitoring.ts
|   |   |— getQueueMetrics(queueName)
|   |   |— getJobCounts(queueName)
|   |   |— getFailedJobs(queueName)
|   |   |— retryFailedJob(jobId)
|   |   |— cleanOldJobs(queueName, age)
|— types.ts
|   |— JobData interface
|   |— JobResult interface
|   |— QueueConfig interface
|   |— WorkerOptions interface
...

```

📊 SAT CATALOG MANAGEMENT

Component 33: SAT Catalog Service

****Purpose:**** Manage and provide access to SAT official catalogs (product codes, tax regimes, etc.).

****Technical Details:****

- Load SAT catalogs from official sources
- Search and filter catalog data
- Cache catalog data in Redis
- Periodic updates from SAT
- Versioning for catalog changes

****Inputs:****

- Catalog type (product codes, tax regimes, units, etc.)
- Search query
- Filters

****Outputs:****

- Catalog entries
- Search results
- Catalog metadata

****Dependencies:****

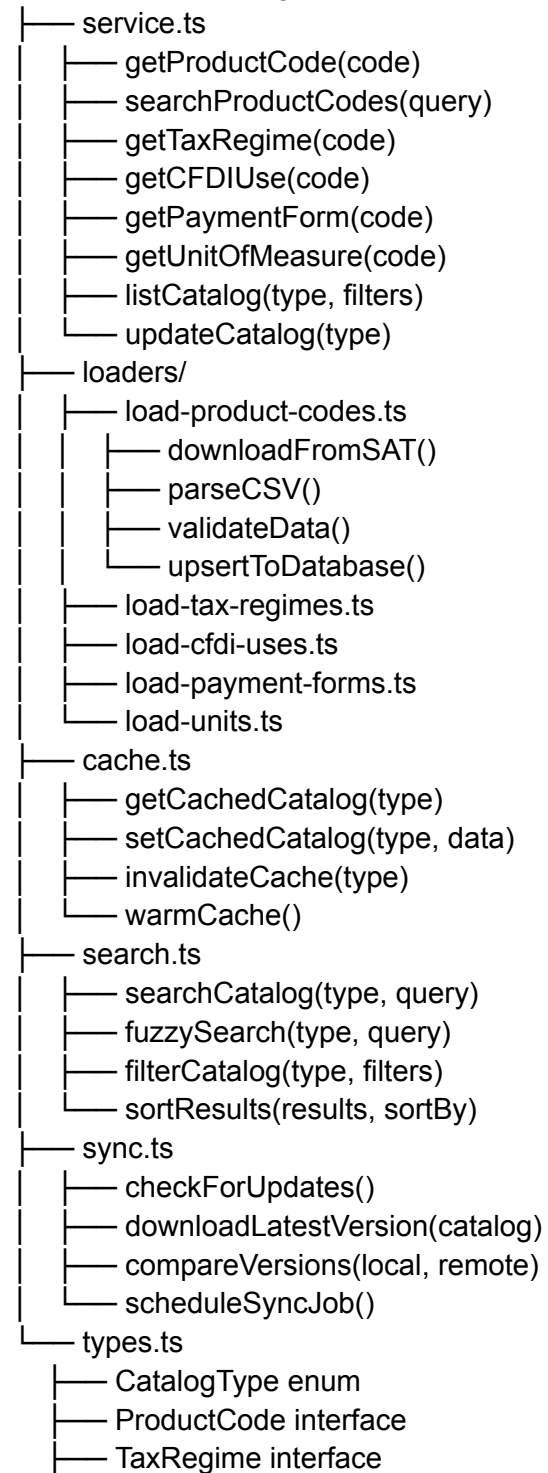
- PostgreSQL (SAT catalog tables)
- Redis (catalog cache)

- SAT official website (for updates)
- AI service (for intelligent search)

****Files/Modules:****

...

src/server/sat-catalog/



```

|— CFDIUse interface
|— PaymentForm interface
|— Unit interface
...

```

🔍 SEARCH & FILTERING

Component 34: Search Service

****Purpose:**** Provide full-text search across invoices, customers, products, etc.

****Technical Details:****

- PostgreSQL full-text search
- Elasticsearch integration (optional for scale)
- Multi-entity search
- Search suggestions
- Recent searches
- Search filters and facets

****Inputs:****

- Search query
- Entity types to search
- Filters
- Pagination

****Outputs:****

- Search results (ranked)
- Result count
- Search suggestions
- Facets

****Dependencies:****

- PostgreSQL (with tsvector)
- Redis (search cache)
- Elasticsearch (optional)

****Files/Modules:****

...

src/server/search/

```

|— service.ts
|
|   |— search(orgId, query, types, filters)
|   |— searchInvoices(orgId, query, filters)

```

```

|— searchCustomers(orgId, query)
|— searchProducts(orgId, query)
|— searchExpenses(orgId, query)
|— getSuggestions(query)
|— getRecentSearches(userId)
|— indexing/
|   |— index-invoice.ts
|   |   |— indexInvoice(invoice)
|   |   |— updateInvoiceIndex(invoiceId)
|   |   |— deleteInvoiceIndex(invoiceId)
|   |— index-customer.ts
|   |— index-product.ts
|   |— index-expense.ts
|— postgres-search.ts
|   |— createSearchVector(text)
|   |— executeSearch(query, table, fields)
|   |— rankResults(results)
|   |— highlightMatches(text, query)
|— elasticsearch.ts
|   |— ESSearchService class
|   |— indexDocument(index, id, document)
|   |— search(index, query, filters)
|   |— bulkIndex(documents)
|   |— deleteIndex(index)
|— suggestions.ts
|   |— getSuggestions(query, orgId)
|   |— getPopularSearches(orgId)
|   |— saveRecentSearch(userId, query)
|   |— clearRecentSearches(userId)
|— types.ts
|   |— SearchResult interface
|   |— SearchFilters interface
|   |— SearchSuggestion interface
|   |— Facet interface
...

```

ANALYTICS & REPORTING

Component 35: Analytics Service

****Purpose:**** Provide business intelligence and analytics dashboards.

****Technical Details:****

- Key metrics calculation (revenue, expenses, profit)
- Time-series data aggregation
- Comparison periods (MoM, YoY)
- Data visualization preparation
- Custom reports
- Dashboard widgets

****Inputs:****

- Organization ID
- Date range
- Metric types
- Comparison period

****Outputs:****

- Metric values
- Time-series data
- Comparison data
- Chart data

****Dependencies:****

- PostgreSQL (with window functions)
- Redis (metrics cache)
- Job queue (for heavy calculations)

****Files/Modules:****

...

src/server/analytics/

```
|— service.ts
|   |— getDashboardMetrics(orgId, dateRange)
|   |— getRevenueMetrics(orgId, dateRange)
|   |— getExpenseMetrics(orgId, dateRange)
|   |— getProfitMargin(orgId, dateRange)
|   |— getCustomerMetrics(orgId, dateRange)
|   |— getInvoiceMetrics(orgId, dateRange)
|   |— getComparison(orgId, current, previous)
|— metrics/
|   |— revenue.ts
|       |— calculateTotalRevenue(orgId, start, end)
|       |— calculateMonthlyRevenue(orgId, year)
|       |— calculateRevenueByCustomer(orgId, start, end)
|       |— calculateRevenueGrowth(current, previous)
|   |— expenses.ts
|       |— calculateTotalExpenses(orgId, start, end)
```

```
├── calculateExpensesByCategory(orgId, start, end)
├── calculateMonthlyExpenses(orgId, year)
├── calculateExpenseGrowth(current, previous)
├── profit.ts
│   ├── calculateGrossProfit(revenue, cogs)
│   ├── calculateNetProfit(revenue, expenses)
│   ├── calculateProfitMargin(profit, revenue)
│   └── calculateEBITDA(orgId, start, end)
├── customers.ts
│   ├── calculateCustomerCount(orgId)
│   ├── calculateNewCustomers(orgId, start, end)
│   ├── calculateCustomerRetention(orgId, period)
│   ├── calculateCustomerLifetimeValue(customerId)
│   └── calculateTopCustomers(orgId, start, end, limit)
├── invoices.ts
│   ├── calculateInvoiceCount(orgId, start, end)
│   ├── calculateAverageInvoiceValue(orgId, start, end)
│   ├── calculateCollectionRate(orgId, start, end)
│   ├── calculateDaysToPayment(orgId, start, end)
│   └── calculateOverdueAmount(orgId)
├── aggregation.ts
│   ├── aggregateByDay(data)
│   ├── aggregateByWeek(data)
│   ├── aggregateByMonth(data)
│   ├── aggregateByQuarter(data)
│   └── aggregateByYear(data)
├── comparison.ts
│   ├── compareWithPreviousPeriod(current, previous)
│   ├── calculateGrowthRate(current, previous)
│   ├── calculatePercentageChange(current, previous)
│   └── formatComparison(current, previous)
├── cache.ts
│   ├── getCachedMetrics(key)
│   ├── setCachedMetrics(key, data, ttl)
│   ├── invalidateMetricsCache(orgId)
│   └── warmMetricsCache(orgId)
├── types.ts
│   ├── Metric interface
│   ├── TimeSeriesData interface
│   ├── Comparison interface
│   └── DashboardData interface
...
---
```

🛡️ SECURITY & COMPLIANCE

Component 36: Encryption Service

****Purpose:**** Handle encryption and decryption of sensitive data.

****Technical Details:****

- AES-256 encryption for data at rest
- Encryption key management
- Encrypt CSD private keys
- Encrypt PAC credentials
- Encrypt sensitive fields in database

****Inputs:****

- Plain text data
- Encryption key (from env or KMS)

****Outputs:****

- Encrypted data (base64)
- Decrypted data

****Dependencies:****

- crypto library
- Key management system (AWS KMS, HashiCorp Vault, or env vars)

****Files/Modules:****

...

src/server/security/

```
|— encryption.ts
|   |— encrypt(data, key)
|   |— decrypt(encryptedData, key)
|   |— encryptField(value)
|   |— decryptField(encryptedValue)
|   |— generateKey()
|   |— rotateKey(oldKey, newKey)
|— key-management.ts
|   |— getEncryptionKey()
|   |— getKeyFromKMS()
|   |— getKeyFromVault()
|   |— storeKey(key)
|   |— validateKey(key)
|— hashing.ts
|   |— hashPassword(password)
```



```
| |— verifyPassword(password, hash)
| |— hashData(data)
| |— compareHash(data, hash)
|— types.ts
| |— EncryptionConfig interface
| |— KeyMetadata interface
...
---
```

Component 37: Audit Log Service

****Purpose:**** Track all user actions and system events for compliance and debugging.

****Technical Details:****

- Log all CRUD operations
- Track user actions with context
- IP address and user agent logging
- Change tracking (old vs. new values)
- Retention policies
- Search and filter audit logs

****Inputs:****

- User ID
- Action type
- Entity type and ID
- Old and new values
- Request context

****Outputs:****

- Audit log entry
- Audit log history
- Filtered audit logs

****Dependencies:****

- PostgreSQL (activity_log table)
- Request middleware

****Files/Modules:****

...

src/server/audit/

```
|— service.ts
| |— logAction(userId, action, entity, data)
| |— logCreate(userId, entity, values)
```

```

├── logUpdate(userId, entity, oldValues, newValues)
├── logDelete(userId, entity, values)
├── getAuditLog(filters, pagination)
├── getUserActivity(userId, start, end)
├── getEntityHistory(entityType, entityId)
├── middleware.ts
│   ├── auditMiddleware()
│   ├── extractContext(req)
│   ├── shouldAudit(action)
│   └── logRequest(req, res)
├── filters.ts
│   ├── filterByUser(userId)
│   ├── filterByEntity(entityType, entityId)
│   ├── filterByAction(action)
│   ├── filterByDateRange(start, end)
│   └── buildQuery(filters)
├── retention.ts
│   ├── cleanOldLogs(retentionDays)
│   ├── archiveLogs(olderThan)
│   └── scheduleCleanup()
├── types.ts
│   ├── AuditLog interface
│   ├── AuditAction enum
│   └── AuditFilters interface
...

```

Component 38: Rate Limiting Service

****Purpose:**** Prevent abuse and ensure fair usage of API resources.

****Technical Details:****

- Rate limiting by IP and user
- Different limits for different endpoints
- Redis-based rate limit tracking
- Sliding window algorithm
- Rate limit headers in responses

****Inputs:****

- Request identifier (IP, user ID, API key)
- Endpoint or action
- Rate limit configuration

****Outputs:****

- Allow or deny request
- Remaining quota
- Reset time

****Dependencies:****

- Redis
- Express middleware

****Files/Modules:****

...

src/server/rate-limiting/

```
|— service.ts
|   |— checkRateLimit(identifier, action)
|   |— incrementCounter(identifier, action)
|   |— getRemainingQuota(identifier, action)
|   |— getResetTime(identifier, action)
|   |— resetLimit(identifier, action)
|— middleware.ts
|   |— rateLimitMiddleware(config)
|   |— extractIdentifier(req)
|   |— setRateLimitHeaders(res, quota)
|   |— handleRateLimitExceeded(req, res)
|— strategies/
|   |— fixed-window.ts
|   |— sliding-window.ts
|   |— token-bucket.ts
|— config.ts
|   |— API_RATE_LIMITS
|   |— AUTH_RATE_LIMITS
|   |— WEBHOOK_RATE_LIMITS
|   |— getRateLimitConfig(action)
|— types.ts
|   |— RateLimitConfig interface
|   |— RateLimitResult interface
|   |— RateLimitStrategy enum
|— ...
```

API & INTEGRATION

Component 39: REST API Layer

****Purpose:**** Expose RESTful API endpoints for all application functionality.

****Technical Details:****

- RESTful design principles
- OpenAPI/Swagger documentation
- Versioning (v1, v2, etc.)
- Request validation
- Response formatting
- Error handling

****Inputs:****

- HTTP requests (GET, POST, PUT, DELETE)
- Request parameters, query strings, body

****Outputs:****

- JSON responses
- HTTP status codes
- Error messages

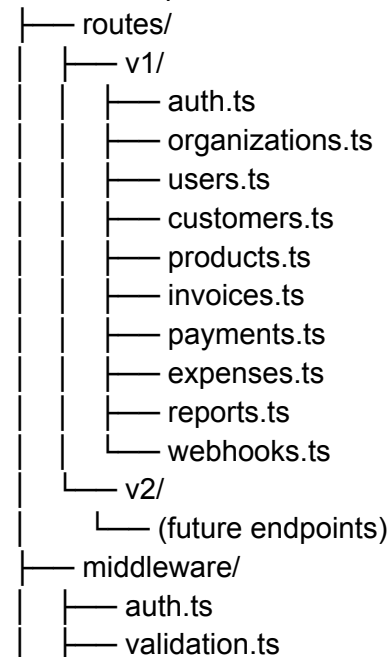
****Dependencies:****

- Express or Next.js API routes
- Request validation library (Zod)
- All service layers

****Files/Modules:****

...

src/server/api/



```

|   |— error-handler.ts
|   |— rate-limit.ts
|   |— cors.ts
|   |— logger.ts
|   |— validators/
|   |   |— invoice-validator.ts
|   |   |— customer-validator.ts
|   |   |— payment-validator.ts
|   |   |— common-validators.ts
|   |— responses/
|   |   |— success.ts
|   |   |— error.ts
|   |   |— paginated.ts
|   |   |— format.ts
|   |— documentation/
|   |   |— openapi.yaml
|   |   |— swagger-setup.ts
|   |   |— examples/
|   |— types.ts
|   |   |— APIRequest interface
|   |   |— APIResponse interface
|   |   |— APIError interface
...

```

Component 40: tRPC API Layer

****Purpose:**** Type-safe API layer for frontend-backend communication.

****Technical Details:****

- End-to-end type safety
- Automatic type inference
- Input validation with Zod
- Middleware for auth and logging
- Subscription support (optional)

****Inputs:****

- Typed procedure calls from frontend

****Outputs:****

- Typed responses
- Type-safe errors

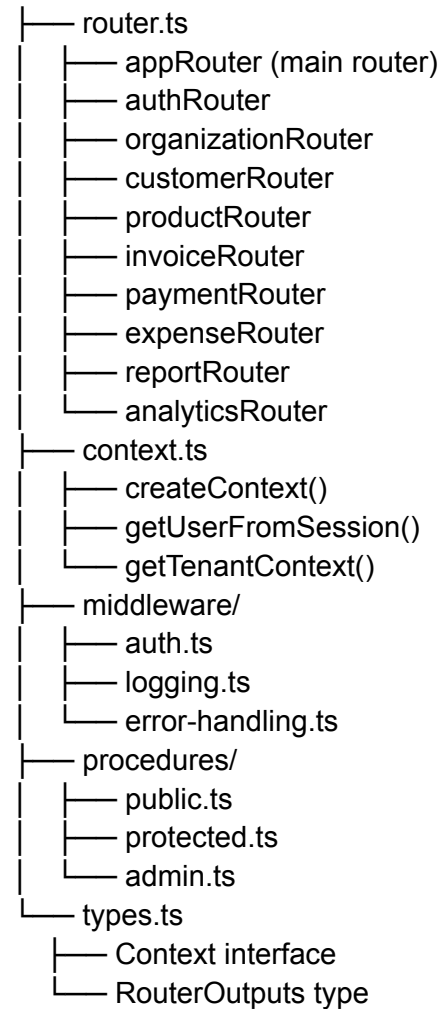
****Dependencies:****

- tRPC library
- Zod
- All service layers

****Files/Modules:****

...

src/server/trpc/



...

Component 41: Webhook Service

****Purpose:**** Send and receive webhooks for integrations.

****Technical Details:****

- Webhook registration and management

- Secure webhook delivery (HMAC signatures)
- Retry logic for failed deliveries
- Webhook event types
- Incoming webhook validation

****Inputs:****

- Webhook URL
- Event types to subscribe
- Webhook secret
- Event data

****Outputs:****

- Delivery status
- Response from webhook endpoint
- Retry status

****Dependencies:****

- HTTP client
- Job queue (for retries)
- PostgreSQL (webhook registrations)

****Files/Modules:****

...

src/server/webhooks/

```

├── service.ts
│   ├── registerWebhook(orgId, url, events, secret)
│   ├── updateWebhook(webhookId, data)
│   ├── deleteWebhook(webhookId)
│   ├── listWebhooks(orgId)
│   ├── sendWebhook(webhookId, event, data)
│   ├── validateWebhook(signature, body, secret)
│   └── getWebhookLogs(webhookId)
├── events.ts
│   ├── WEBHOOK_EVENTS
│   ├── triggerEvent(orgId, event, data)
│   ├── getEventPayload(event, data)
│   └── formatEventData(event, data)
├── delivery.ts
│   ├── deliverWebhook(webhook, payload)
│   ├── signPayload(payload, secret)
│   ├── verifySignature(signature, payload, secret)
│   ├── handleDeliverySuccess(webhook, response)
│   └── handleDeliveryFailure(webhook, error)
└── retry.ts

```

```

|   |— scheduleRetry(webhookId, attempt)
|   |— getRetryDelay(attempt)
|   |— shouldRetry(attempt, error)
|   |— markAsFailed(webhookId)
|— incoming.ts
|   |— handleIncomingWebhook(req, res)
|   |— validateIncoming(req)
|   |— routeWebhook(event, data)
|   |— sendResponse(res, status, data)
|— types.ts
|   |— Webhook interface
|   |— WebhookEvent enum
|   |— WebhookDelivery interface
|   |— WebhookLog interface
...

```

🎨 FRONTEND COMPONENTS

Component 42: UI Component Library

****Purpose:**** Reusable UI components built with Radix UI and Tailwind CSS.

****Technical Details:****

- Design system components
- Shadcn/ui components
- Accessibility (ARIA)
- Dark mode support
- Responsive design

****Components:****

- Button, Input, Select, Checkbox, Radio
- Dialog, Sheet, Popover, Tooltip
- Table, DataTable
- Form components
- Toast notifications

****Files/Modules:****

...

src/components/ui/

```

|— button.tsx
|— input.tsx
|— select.tsx

```



```
|— checkbox.tsx
|— radio-group.tsx
|— dialog.tsx
|— sheet.tsx
|— popover.tsx
|— tooltip.tsx
|— dropdown-menu.tsx
|— table.tsx
|— data-table.tsx
|— form.tsx
|— label.tsx
|— textarea.tsx
|— switch.tsx
|— tabs.tsx
|— card.tsx
|— badge.tsx
|— alert.tsx
|— toast.tsx
|— skeleton.tsx
|— spinner.tsx
|— avatar.tsx
|— calendar.tsx
|— date-picker.tsx
|— command.tsx
...
---
```

Component 43: Dashboard Layout

****Purpose:**** Main application layout with navigation, header, and sidebar.

****Technical Details:****

- Responsive layout
- Collapsible sidebar
- Top navigation bar
- Breadcrumbs
- User menu
- Organization switcher
- Mobile menu

****Files/Modules:****

...

src/components/layout/

```

|— dashboard-layout.tsx
|— sidebar.tsx
|   |— SidebarNav
|   |— SidebarItem
|   |— SidebarCollapse
|— header.tsx
|   |— Header
|   |— UserMenu
|   |— OrganizationSwitcher
|   |— NotificationBell
|— breadcrumbs.tsx
|— mobile-nav.tsx
|— footer.tsx
...

```

Component 44: Invoice Components

****Purpose:**** UI components specific to invoice management.

****Technical Details:****

- Invoice form (multi-step)
- Invoice items table (add/edit/delete rows)
- Invoice preview
- Invoice list with filters
- Invoice status badges
- Invoice actions menu

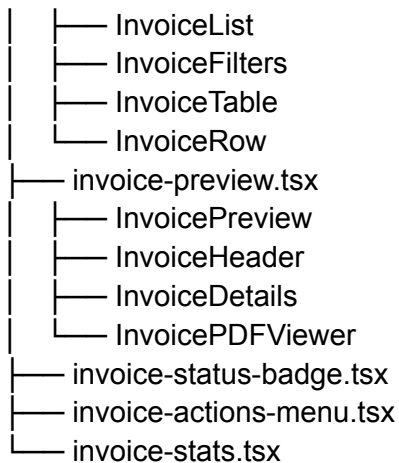
****Files/Modules:****

...

```

src/components/invoices/
|— invoice-form.tsx
|   |— InvoiceForm
|   |— CustomerSelect
|   |— InvoiceItemsTable
|   |— InvoiceTotals
|   |— InvoiceActions
|— invoice-item-row.tsx
|   |— InvoiceItemRow
|   |— ProductSelect
|   |— SATCodeSearch
|   |— ItemCalculations
|— invoice-list.tsx

```



FRONTEND COMPONENTS (Continued)

Component 45: Form Components

Purpose: Complex form components with validation and state management.

Technical Details:

- React Hook Form integration
- Zod validation schemas
- Multi-step forms
- Field arrays (

Component 45: Form Components (Continued)

Purpose: Complex form components with validation and state management.

Technical Details:

- React Hook Form integration
- Zod validation schemas
- Multi-step forms
- Field arrays (dynamic rows)
- Async validation
- Auto-save drafts

- Form state persistence

Files/Modules:

```
src/components/forms/  
├── customer-form.tsx  
│   ├── CustomerForm  
│   ├── RFCField  
│   ├── TaxRegimeSelect  
│   ├── CFDIUseSelect  
│   └── AddressFields  
├── product-form.tsx  
│   ├── ProductForm  
│   ├── SATCodeField  
│   ├── PricingFields  
│   ├── TaxRateField  
│   └── InventoryFields  
├── expense-form.tsx  
│   ├── ExpenseForm  
│   ├── ReceiptUpload  
│   ├── CategorySelect  
│   ├── VendorFields  
│   └── DeductibilityCheck  
├── payment-form.tsx  
│   ├── PaymentForm  
│   ├── AmountField  
│   ├── PaymentMethodSelect  
│   ├── PaymentDatePicker  
│   └── ReferenceField  
├── organization-form.tsx  
│   ├── OrganizationForm  
│   ├── FiscalInfoFields  
│   ├── CertificateUpload  
│   ├── PACConfiguration  
│   └── BillingSettings  
└── form-fields/  
    ├── rfc-input.tsx  
    ├── currency-input.tsx  
    ├── percentage-input.tsx  
    ├── phone-input.tsx  
    ├── email-input.tsx  
    ├── file-upload.tsx  
    ├── image-upload.tsx  
    ├── rich-text-editor.tsx  
    └── date-range-picker.tsx
```

```
|   |___ autocomplete.tsx
|___ validation/
|   |___ customer-schema.ts
|   |___ product-schema.ts
|   |___ invoice-schema.ts
|   |___ expense-schema.ts
|   |___ common-schemas.ts
...

```

Component 46: Data Tables

****Purpose:**** Advanced data tables with sorting, filtering, pagination, and bulk actions.

****Technical Details:****

- TanStack Table (React Table)
- Server-side pagination
- Column sorting
- Global and column filters
- Column visibility toggle
- Row selection
- Bulk actions
- Export functionality
- Responsive design

****Files/Modules:****

...

```
src/components/tables/
|___ data-table.tsx
|   |___ DataTable (generic)
|   |___ TableHeader
|   |___ TableBody
|   |___ TablePagination
|   |___ TableToolbar
|___ columns/
|   |___ invoice-columns.tsx
|   |___ customer-columns.tsx
|   |___ product-columns.tsx
|   |___ expense-columns.tsx
|   |___ payment-columns.tsx
|   |___ column-helpers.tsx
|___ filters/
|   |___ table-filters.tsx

```

```

|   |   | date-range-filter.tsx
|   |   | status-filter.tsx
|   |   | amount-range-filter.tsx
|   |   | search-filter.tsx
|   |   |
|   |   | actions/
|   |   | |   | bulk-actions.tsx
|   |   | |   | row-actions.tsx
|   |   | |   | export-button.tsx
|   |   | |   |
|   |   | |   | hooks/
|   |   | |   | |   | use-table-state.ts
|   |   | |   | |   | use-table-filters.ts
|   |   | |   | |   | use-table-sorting.ts
|   |   | |   |
|   |   | |   | ...
|   |   | |   |
|   |   | |   | ---

```

Component 47: Dashboard Widgets

****Purpose:**** Dashboard components for analytics and key metrics visualization.

****Technical Details:****

- Metric cards (KPIs)
- Charts (line, bar, pie, area)
- Comparison indicators
- Trend indicators
- Real-time updates
- Responsive charts

****Files/Modules:****

...

```

src/components/dashboard/
|   | metric-card.tsx
|   | |   | MetricCard
|   | |   | MetricValue
|   | |   | MetricChange
|   | |   | MetricTrend
|   | |   |
|   | |   | charts/
|   | |   | |   | revenue-chart.tsx
|   | |   | |   | expense-chart.tsx
|   | |   | |   | profit-chart.tsx
|   | |   | |   | invoice-status-chart.tsx
|   | |   | |   | customer-distribution-chart.tsx
|   | |   | |   | chart-container.tsx
|   | |   | |   |
|   | |   | |   | widgets/

```

```

|   |— recent-invoices.tsx
|   |— overdue-invoices.tsx
|   |— top-customers.tsx
|   |— upcoming-payments.tsx
|   |— monthly-summary.tsx
|   |— quick-actions.tsx
|— overview-stats.tsx
|   |— RevenueStats
|   |— ExpenseStats
|   |— ProfitStats
|   |— InvoiceStats
|— filters/
|   |— date-range-selector.tsx
|   |— period-selector.tsx
...

```

Component 48: SAT Code Autocomplete

****Purpose:**** Intelligent autocomplete for SAT product/service codes with AI-powered search.

****Technical Details:****

- Combobox/autocomplete UI
- Debounced API calls
- AI-powered suggestions
- Fuzzy matching
- Recent selections
- Popular codes
- Keyboard navigation

****Files/Modules:****

...

```

src/components/sat/
|— sat-code-search.tsx
|   |— SATCodeSearch
|   |— SearchInput
|   |— SuggestionsList
|   |— CodeDetails
|   |— RecentCodes
|— sat-code-display.tsx
|   |— SATCodeBadge
|   |— CodeInfo
|   |— CategoryLabel

```

```

|— hooks/
|   |— use-sat-search.ts
|   |— use-recent-codes.ts
|   |— use-popular-codes.ts
|— types.ts
|   |— SATCodeSearchResult
|   |— SATCodeSuggestion
...

```

Component 49: File Upload Components

****Purpose:**** File upload UI with drag-and-drop, preview, and validation.

****Technical Details:****

- Drag and drop
- Multiple file upload
- File type validation
- Size validation
- Image preview
- PDF preview
- Progress indicators
- Upload queue

****Files/Modules:****

...

```

src/components/upload/
|— file-upload.tsx
|   |— FileUpload
|   |— DropZone
|   |— FileList
|   |— FileItem
|   |— UploadProgress
|— image-upload.tsx
|   |— ImageUpload
|   |— ImagePreview
|   |— ImageCropper
|   |— ImageEditor
|— receipt-upload.tsx
|   |— ReceiptUpload
|   |— ReceiptScanner
|   |— ReceiptPreview
|   |— ExtractedData

```



```

|— certificate-upload.tsx
|   |— CertificateUpload
|   |— CERFileUpload
|   |— KEYFileUpload
|   |— PasswordInput
|— hooks/
|   |— use-file-upload.ts
|   |— use-image-upload.ts
|   |— use-drag-drop.ts
...

```

Component 50: Notification Components

****Purpose:**** In-app notification system with toast messages and notification center.

****Technical Details:****

- Toast notifications
- Notification center/inbox
- Real-time updates
- Notification preferences
- Action buttons in notifications
- Grouped notifications

****Files/Modules:****

...

src/components/notifications/

```

|— toast-provider.tsx
|   |— ToastProvider
|   |— Toast
|   |— ToastTitle
|   |— ToastDescription
|   |— ToastAction
|— notification-center.tsx
|   |— NotificationCenter
|   |— NotificationList
|   |— NotificationItem
|   |— NotificationFilters
|   |— MarkAllRead
|— notification-bell.tsx
|   |— NotificationBell
|   |— UnreadBadge
|   |— NotificationDropdown

```

```
└─ hooks/
   └─ use-notifications.ts
   └─ use-toast.ts
   └─ use-notification-subscription.ts
...
```

Component 51: Modal & Dialog Components

Purpose: Modals and dialogs for confirmations, forms, and detail views.

Technical Details:

- Confirmation dialogs
- Form modals
- Detail view drawers
- Multi-step modals
- Fullscreen modals
- Loading states

Files/Modules:

...

```
src/components/modals/
├─ confirmation-dialog.tsx
│   ├── ConfirmationDialog
│   ├── AlertDialog
│   └─ DestructiveDialog
├─ invoice-preview-modal.tsx
│   ├── InvoicePreviewModal
│   ├── XMLViewer
│   ├── PDFViewer
│   └─ DownloadButtons
├─ customer-detail-drawer.tsx
│   ├── CustomerDetailDrawer
│   ├── CustomerInfo
│   ├── CustomerInvoices
│   └─ CustomerStats
├─ payment-modal.tsx
│   ├── PaymentModal
│   ├── PaymentForm
│   └─ PaymentConfirmation
├─ cancel-invoice-modal.tsx
│   ├── CancelInvoiceModal
│   └─ CancellationReason
```

```

|   |   └─ RelatedInvoices
|   └─ hooks/
|       └─ use-modal.ts
|       └─ use-confirm.ts
...

```

🦿 INTEGRATION COMPONENTS

Component 52: External SAT API Integration

****Purpose:**** Integrate with SAT's official APIs for RFC validation, invoice status, etc.

****Technical Details:****

- RFC validation API
- Invoice status query (by UUID)
- CFDI cancellation API
- Certificate validation
- Rate limiting and retries
- Error handling for SAT APIs

****Inputs:****

- RFC (for validation)
- UUID (for invoice status)
- Organization credentials

****Outputs:****

- RFC validation result
- Invoice status
- Cancellation status

****Dependencies:****

- SAT API credentials
- SOAP or REST client
- XML parser

****Files/Modules:****

...

```

src/server/integrations/sat/
├── api-client.ts
├── └── SATAPIClient class
├── └── validateRFC(rfc)
├── └── queryInvoiceStatus(uuid)

```

```

|   |— requestCancellation(uuid, reason)
|   |— validateCertificate(cert)
|   |— handleAPIError(error)
|— rfc-validation.ts
|   |— validateRFCWithSAT(rfc)
|   |— getRFCDetails(rfc)
|   |— checkRFCStatus(rfc)
|   |— parseRFCResponse(response)
|— invoice-status.ts
|   |— queryInvoiceStatus(uuid)
|   |— getInvoiceDetails(uuid)
|   |— checkCancellationStatus(uuid)
|   |— parseStatusResponse(response)
|— cancellation.ts
|   |— requestCancellation(uuid, rfc, reason)
|   |— confirmCancellation(uuid)
|   |— getCancellationStatus(uuid)
|   |— handleCancellationResponse(response)
|— types.ts
|   |— RFCValidationResult interface
|   |— InvoiceStatus interface
|   |— CancellationResult interface
...

```

Component 53: Payment Gateway Integration (Stripe)

****Purpose:**** Full Stripe integration for payment processing and subscription management.

****Technical Details:****

- Stripe Elements for card input
- Payment intents
- Customer management
- Subscription billing
- Webhook handling
- 3D Secure (SCA) support

****Inputs:****

- Payment amount and currency
- Customer information
- Payment method
- Subscription plan

****Outputs:****

- Payment status
- Transaction ID
- Payment method saved
- Subscription status

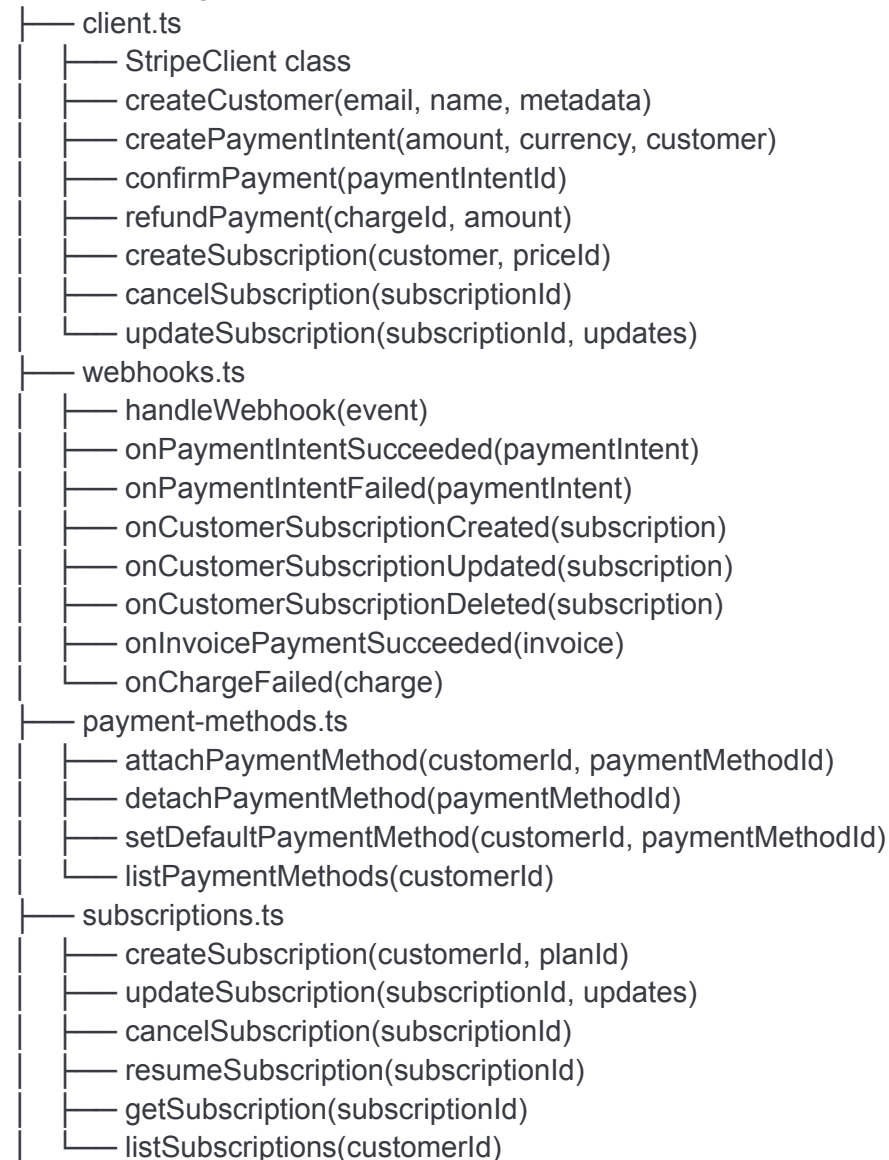
****Dependencies:****

- Stripe SDK
- Webhook endpoint
- Payment service

****Files/Modules:****

...

src/server/integrations/stripe/



```

└─ types.ts
  └─ StripeCustomer interface
  └─ StripePaymentIntent interface
  └─ StripeSubscription interface
  └─ StripeWebhookEvent interface
...

```

Component 54: Email Service Integration (SendGrid/Postmark)

Purpose: Transactional email sending through third-party email service.

Technical Details:

- SMTP or API integration
- Template rendering
- Dynamic content
- Attachment support
- Email tracking
- Bounce handling

Inputs:

- Recipient email
- Template ID or HTML
- Template variables
- Attachments

Outputs:

- Email ID
- Delivery status
- Tracking data

Dependencies:

- SendGrid or Postmark API
- Email template renderer

Files/Modules:

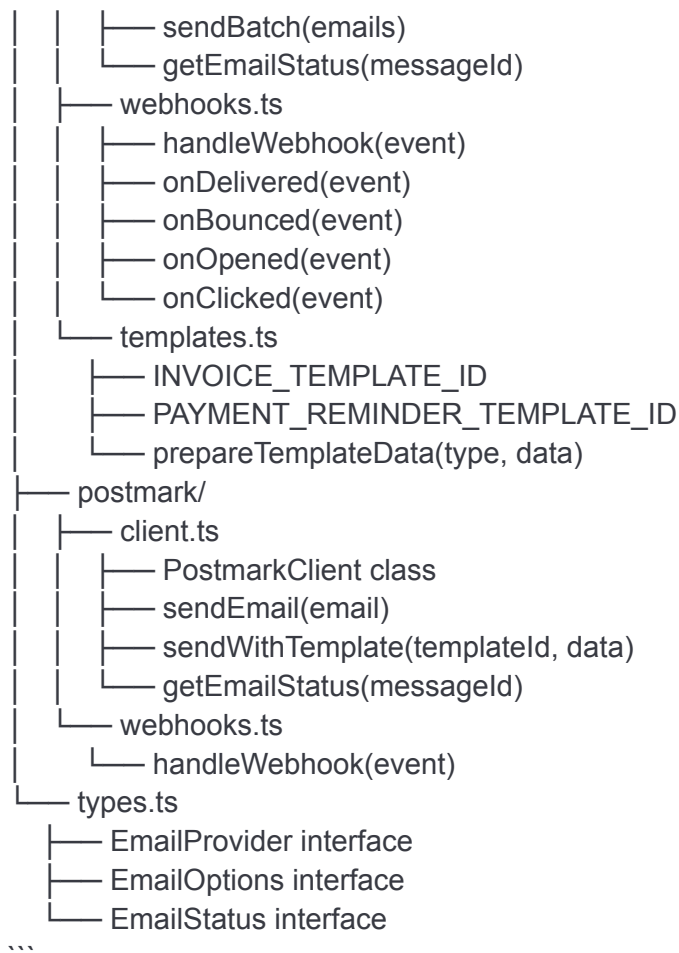
...

src/server/integrations/email/

```

└─ sendgrid/
  └─ client.ts
    └─ SendGridClient class
    └─ sendEmail(to, subject, html, text)
    └─ sendWithTemplate(to, templateId, data)

```



■■■■■

Component 55: Cloud Storage Integration (S3/R2)

****Purpose:**** Store files (XMLs, PDFs, receipts) in cloud object storage.

****Technical Details:****

- S3-compatible API integration
- Presigned URLs for secure access
- Multipart upload for large files
- Lifecycle policies
- CDN integration

```

**Inputs:**

```

- File buffer or stream
- File metadata
- Bucket and path

****Outputs:****

- File URL
- File key
- Presigned URL

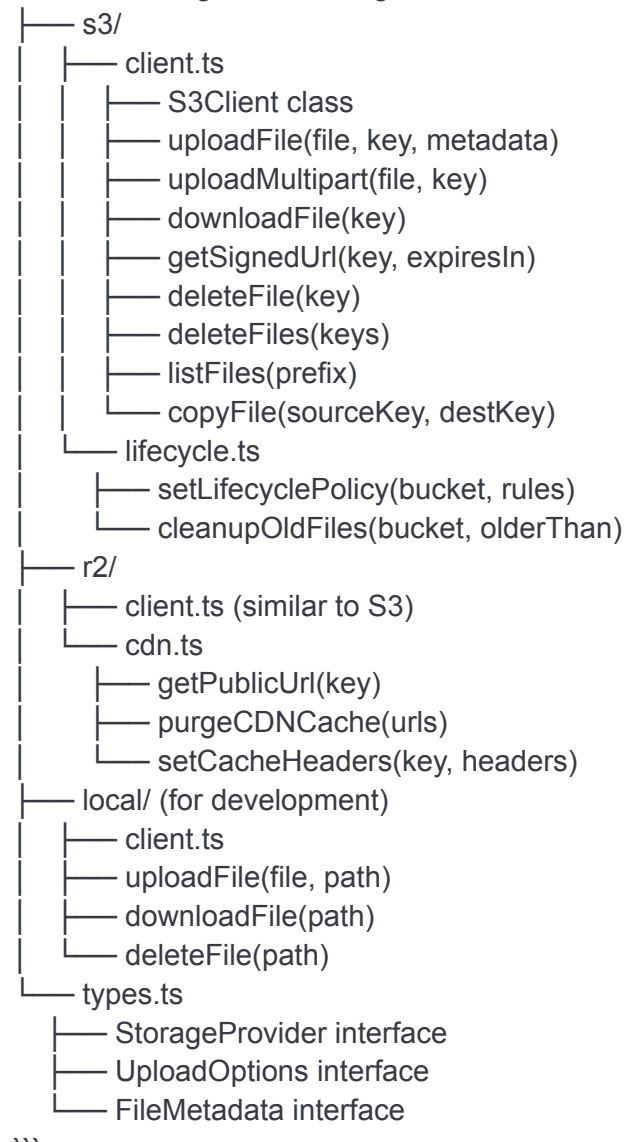
****Dependencies:****

- AWS SDK or compatible library
- Storage credentials

****Files/Modules:****

...

src/server/integrations/storage/



🖋️ TESTING COMPONENTS

Component 56: Unit Tests

****Purpose:**** Test individual functions and components in isolation.

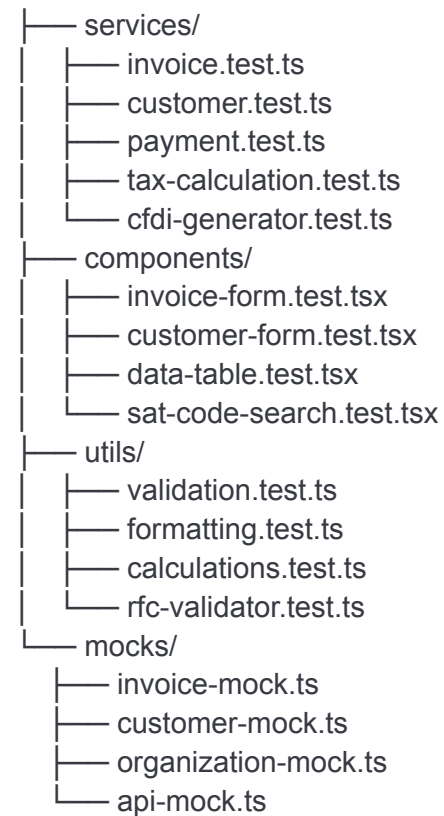
****Technical Details:****

- Jest testing framework
- React Testing Library for components
- Mock services and APIs
- Code coverage
- Snapshot testing

****Files/Modules:****

...

tests/unit/



...

Component 57: Integration Tests

****Purpose:**** Test interactions between multiple components and services.

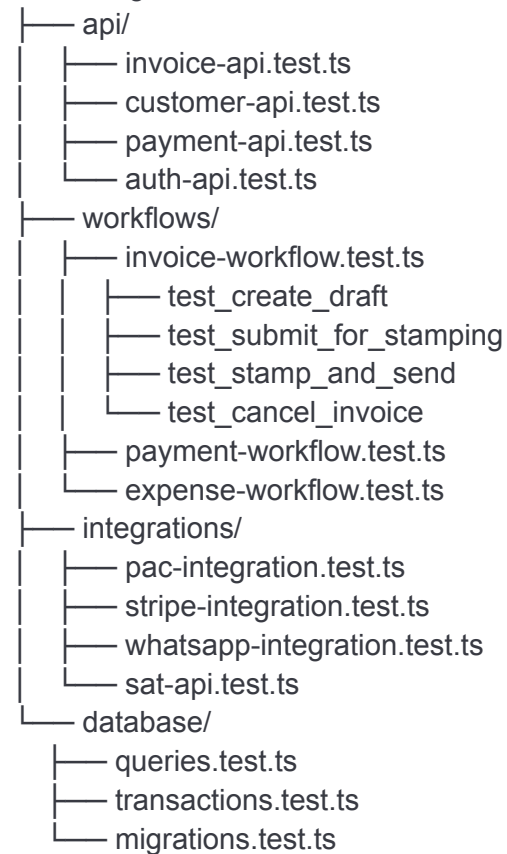
****Technical Details:****

- Test API endpoints
- Test database operations
- Test external integrations
- Test workflows (invoice creation to stamping)

****Files/Modules:****

...

tests/integration/



...

Component 58: End-to-End Tests

****Purpose:**** Test complete user flows from UI to database.

****Technical Details:****

- Playwright or Cypress
- Test user journeys

- Visual regression testing
- Cross-browser testing

****Files/Modules:****

...

tests/e2e/

```

├── auth/
│   ├── login.spec.ts
│   ├── registration.spec.ts
│   └── password-reset.spec.ts
├── invoices/
│   ├── create-invoice.spec.ts
│   ├── edit-invoice.spec.ts
│   ├── send-invoice.spec.ts
│   └── cancel-invoice.spec.ts
├── customers/
│   ├── create-customer.spec.ts
│   └── manage-customers.spec.ts
├── payments/
│   ├── record-payment.spec.ts
│   └── online-payment.spec.ts
└── reports/
    ├── generate-report.spec.ts
    └── export-report.spec.ts

```

...

DEPLOYMENT & INFRASTRUCTURE

Component 59: CI/CD Pipeline

****Purpose:**** Automated testing, building, and deployment.

****Technical Details:****

- GitHub Actions or GitLab CI
- Automated testing on PR
- Build and deploy on merge
- Environment-specific deployments
- Rollback capabilities

****Files/Modules:****

...

.github/workflows/

```

├── ci.yml
│   ├── Lint code
│   ├── Run unit tests
│   ├── Run integration tests
│   ├── Check test coverage
│   └── Build application
├── deploy-staging.yml
│   ├── Build Docker images
│   ├── Push to registry
│   ├── Deploy to staging
│   └── Run smoke tests
├── deploy-production.yml
│   ├── Build Docker images
│   ├── Push to registry
│   ├── Deploy to production
│   ├── Run smoke tests
│   └── Notify team
└── e2e-tests.yml
    ├── Run E2E tests nightly
    └── Report failures
...

```

Component 60: Database Migrations

****Purpose:**** Version-controlled database schema changes.

****Technical Details:****

- Migration tool (Prisma, node-pg-migrate, or Flyway)
- Up and down migrations
- Seed data scripts
- Migration versioning

****Files/Modules:****

...

```

packages/database/migrations/
├── 001_initial_schema.sql
├── 002_add_expenses_table.sql
├── 003_add_tax_periods_table.sql
├── 004_add_whatsapp_tables.sql
├── 005_add_analytics_views.sql
└── seeds/
    └── sat_catalogs.sql

```

```
├── tax_regimes.sql
├── default_accounts.sql
└── demo_data.sql
...
---
```

Component 61: Environment Configuration

****Purpose:**** Manage configuration across different environments.

****Technical Details:****

- Environment variables
- Secrets management
- Feature flags
- Configuration validation

****Files/Modules:****

```
...
config/
├── .env.example
├── .env.development
├── .env.staging
├── .env.production
├── env.ts
│   ├── parseEnv()
│   ├── validateEnv()
│   └── getConfig()
├── feature-flags.ts
│   ├── FEATURE_FLAGS
│   ├── isFeatureEnabled(flag)
│   └── getFeatureConfig(flag)
└── secrets/
    ├── encryption-keys/
    ├── api-credentials/
    └── certificates/
...
---
```

Component 62: Monitoring & Logging

****Purpose:**** Application monitoring, error tracking, and logging.

****Technical Details:****

- Error tracking (Sentry)
- Application performance monitoring
- Log aggregation
- Metrics dashboards
- Alerting

```

**Files/Modules:**

```

///

```
src/server/monitoring/
```

```

├── sentry.ts
│   ├── initSentry()
│   ├── captureError(error, context)
│   ├── captureMessage(message, level)
│   └── setUser(user)
├── logger.ts
│   ├── Logger class
│   ├── info(message, metadata)
│   ├── error(message, error, metadata)
│   ├── warn(message, metadata)
│   └── debug(message, metadata)
├── metrics.ts
│   ├── trackMetric(name, value, tags)
│   ├── incrementCounter(name, tags)
│   ├── recordHistogram(name, value, tags)
│   └── setGauge(name, value, tags)
├── health-check.ts
│   ├── checkHealth()
│   ├── checkDatabase()
│   ├── checkRedis()
│   ├── checkExternalAPIs()
│   └── getHealthStatus()
└── alerts.ts
    ├── sendAlert(type, message, severity)
    ├── alertOnError(error)
    └── alertOnThreshold(metric, threshold)

```

///

— — —

Component 63: Docker Configuration

****Purpose:**** Containerize application for consistent deployment.

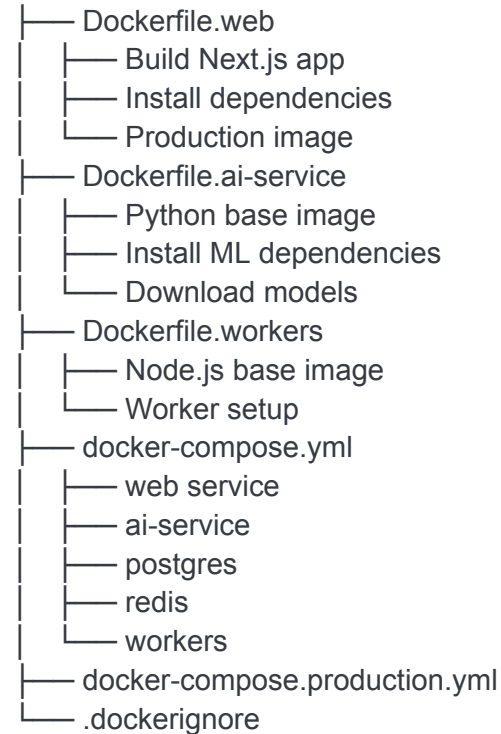
****Technical Details:****

- Multi-stage Docker builds
- Development and production images
- Docker Compose for local development
- Container optimization

****Files/Modules:****

...

docker/



...

Component 64: Infrastructure as Code

****Purpose:**** Define and provision infrastructure using code.

****Technical Details:****

- Terraform or Pulumi
- Cloud provider resources (AWS, GCP, Vercel)
- Network configuration
- Security groups
- Auto-scaling

****Files/Modules:****

```

...
infrastructure/
├── terraform/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   ├── modules/
│   │   ├── database/
│   │   ├── redis/
│   │   ├── storage/
│   │   ├── compute/
│   │   └── networking/
│   └── environments/
│       ├── development/
│       ├── staging/
│       └── production/
└── scripts/
    ├── setup-infrastructure.sh
    ├── deploy.sh
    └── rollback.sh
...

```

DOCUMENTATION COMPONENTS

Component 65: API Documentation

****Purpose:**** Comprehensive API documentation for developers.

****Technical Details:****

- OpenAPI/Swagger spec
- Interactive API explorer
- Code examples
- Authentication guide

****Files/Modules:****

```

...
docs/api/
├── openapi.yaml
├── README.md
├── authentication.md
├── endpoints/
│   └── invoices.md

```



```
| |— customers.md
| |— payments.md
| |— webhooks.md
|— examples/
| |— create-invoice.md
| |— record-payment.md
| |— webhook-handling.md
|— sdks/
| |— javascript.md
| |— python.md
| |— curl.md
...
---
```

Component 66: User Documentation

Purpose: End-user guides and tutorials.

Technical Details:

- Getting started guide
- Feature tutorials
- FAQ
- Video tutorials (optional)
- Troubleshooting

Files/Modules:

```
...
docs/user/
|— getting-started/
| |— registration.md
| |— setup.md
| |— first-invoice.md
|— features/
| |— invoices.md
| |— customers.md
| |— payments.md
| |— expenses.md
| |— reports.md
| |— whatsapp.md
|— tutorials/
| |— monthly-tax-filing.md
| |— expense-tracking.md
| |— payment-collection.md
```

```
|— faq.md
|— troubleshooting.md
...
```

🎯 UTILITY COMPONENTS

Component 67: Validation Utilities

****Purpose:**** Reusable validation functions and schemas.

****Technical Details:****

- Zod schemas
- Custom validators
- RFC validation
- CURP validation
- Email validation
- Phone validation

****Files/Modules:****

...

src/lib/validation/

```
|— schemas/
|   |— invoice-schema.ts
|   |— customer-schema.ts
|   |— product-schema.ts
|   |— payment-schema.ts
|   |— organization-schema.ts
|— validators/
|   |— rfc-validator.ts
|   |— curp-validator.ts
|   |— email-validator.ts
|   |— phone-validator.ts
|   |— amount-validator.ts
|   |— date-validator.ts
|— sanitizers/
|   |— sanitize-input.ts
|   |— sanitize-html.ts
|   |— sanitize-filename.ts
|— types.ts
|   |— ValidationResult interface
|   |— ValidationError interface
...
```

Component 68: Formatting Utilities

****Purpose:**** Format data for display (currency, dates, numbers, etc.).

****Technical Details:****

- Currency formatting (MXN, USD, etc.)
- Date formatting (Spanish/English)
- Number formatting
- RFC formatting
- Phone formatting

****Files/Modules:****

...

src/lib/formatting/

```
|— currency.ts
|   |— formatCurrency(amount, currency, locale)
|   |— parseCurrency(formatted)
|   |— convertCurrency(amount, from, to, rate)
|— date.ts
|   |— formatDate(date, format, locale)
|   |— formatRelativeDate(date)
|   |— parseDate(dateString, format)
|   |— getDateRange(period)
|— number.ts
|   |— formatNumber(number, decimals, locale)
|   |— formatPercentage(value, decimals)
|   |— formatFileSize(bytes)
|   |— parseNumber(formatted)
|— text.ts
|   |— formatRFC(rfc)
|   |— formatPhone(phone, country)
|   |— truncate(text, length)
|   |— capitalize(text)
|   |— slugify(text)
|— invoice.ts
|   |— formatInvoiceNumber(invoice)
|   |— formatInvoiceStatus(status)
|   |— formatPaymentMethod(method)
...
```

Component 69: Calculation Utilities

****Purpose:**** Business logic calculations (tax, totals, etc.).

****Technical Details:****

- Invoice calculations
- Tax calculations
- Discount calculations
- Payment allocations
- Interest calculations

```
**Files/Modules:**
```

///

src/lib/calculations/

- invoice.ts
 - calculateSubtotal(items)
 - calculateDiscount(subtotal, discountRate)
 - calculateTax(subtotal, taxRate)
 - calculateTotal(subtotal, discount, tax)
 - calculateLineItem(item)
- tax.ts
 - calculateIVA(amount)
 - calculateISR(income, regime)
 - calculateWithholding(amount, rate)
 - calculateTaxBase(subtotal, discount)
 - applyTaxRates(amount, rates)
- payment.ts
 - allocatePayment(payment, invoices)
 - calculateOutstanding(invoice, payments)
 - calculateInterest(amount, days, rate)
 - calculatePaymentStatus(invoice, payments)
- financial.ts
 - calculateProfitMargin(revenue, cost)
 - calculateROI(gain, cost)
 - calculateGrowthRate(current, previous)
 - calculateRunRate(amount, period)
- types.ts
 - CalculationResult interface
 - TaxCalculation interface

///

Component 70: Date & Time Utilities

****Purpose:**** Date manipulation and timezone handling.

****Technical Details:****

- Timezone conversions
- Business days calculations
- Due date calculations
- Period helpers (month, quarter, year)

****Files/Modules:****

...

src/lib/datetime/

```
|— timezone.ts
|   |— convertToTimezone(date, timezone)
|   |— getUserTimezone(userId)
|   |— formatInTimezone(date, timezone, format)
|   |— getCurrentTime(timezone)
|— business-days.ts
|   |— addBusinessDays(date, days)
|   |— subtractBusinessDays(date, days)
|   |— isBusinessDay(date)
|   |— getBusinessDaysBetween(start, end)
|   |— getNextBusinessDay(date)
|— periods.ts
|   |— getMonthRange(year, month)
|   |— getQuarterRange(year, quarter)
|   |— getYearRange(year)
|   |— getCurrentPeriod(type)
|   |— getPreviousPeriod(period)
|— due-dates.ts
|   |— calculateDueDate(issueDate, terms)
|   |— isOverdue(dueDate)
|   |— getDaysOverdue(dueDate)
|   |— getUpcomingDueDates(orgId, days)
|— holidays.ts
|   |— MEXICO_HOLIDAYS
|   |— isHoliday(date, country)
|   |— getNextNonHoliday(date)
```

...

Component 71: Constants & Enums

****Purpose:**** Centralized constants and enumerations.

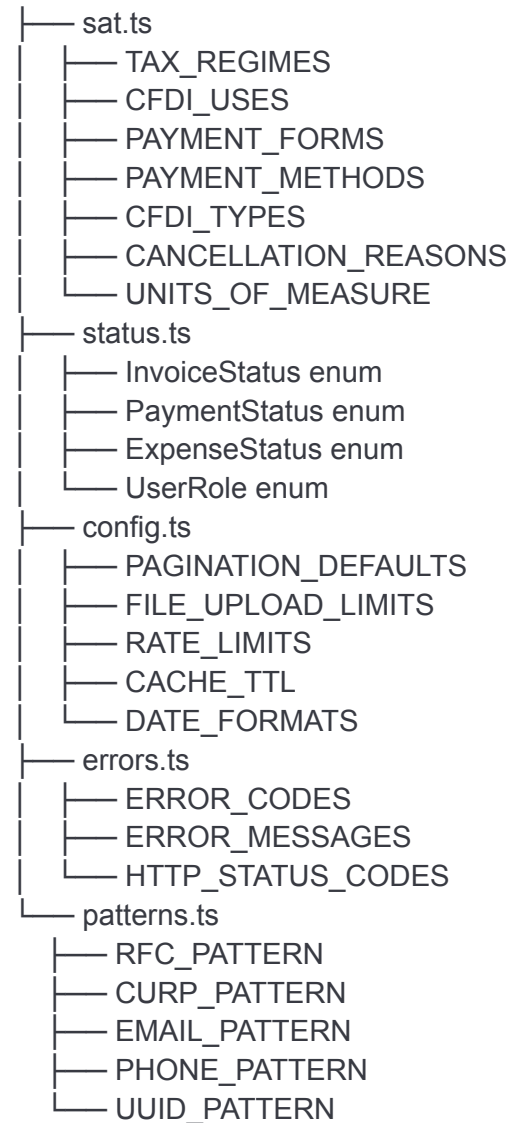
****Technical Details:****

- SAT catalog codes
- Status enums
- Configuration constants
- Error codes
- Regex patterns

****Files/Modules:****

...

src/lib/constants/



...

Component 72: Error Handling Utilities

****Purpose:**** Standardized error handling and custom error classes.

****Technical Details:****

- Custom error classes
- Error formatting
- Error logging
- Error recovery strategies

****Files/Modules:****

...

src/lib/errors/

- custom-errors.ts
 - AppError (base class)
 - ValidationError
 - AuthenticationError
 - AuthorizationError
 - NotFoundError
 - ConflictError
 - ExternalAPIError
 - CFDIError
 - PACError
- error-handler.ts
 - handleError(error, context)
 - formatError(error)
 - isOperationalError(error)
 - sendErrorResponse(res, error)
 - logError(error, context)
- error-recovery.ts
 - retryOperation(operation, config)
 - fallbackValue(error, fallback)
 - recoverFromError(error, recovery)
 - handleGracefulDegradation(error)
- types.ts
 - ErrorContext interface
 - ErrorResponse interface
 - RetryConfig interface

...

🛠️ DEVELOPMENT TOOLS

Component 73: Development Scripts

****Purpose:**** Helper scripts for development tasks.

****Technical Details:****

- Database seeding
- SAT catalog updates
- Test data generation
- Cache clearing
- Migration helpers

****Files/Modules:****

...

scripts/

```
|— dev/
|   |— seed-database.ts
|   |— reset-database.ts
|   |— generate-test-data.ts
|   |— clear-cache.ts
|   |— update-sat-catalog.ts
|— setup/
|   |— setup-local.sh
|   |— install-dependencies.sh
|   |— setup-database.sh
|   |— setup-ai-service.sh
|— migration/
|   |— create-migration.ts
|   |— run-migrations.ts
|   |— rollback-migration.ts
|   |— generate-migration-docs.ts
|— utils/
|   |— backup-database.sh
|   |— restore-database.sh
|   |— export-data.ts
|   |— import-data.ts
```

...

Component 74: Development Tools Configuration

****Purpose:**** Configure development tools (linters, formatters, etc.).

****Technical Details:****

- ESLint configuration
- Prettier configuration
- TypeScript configuration
- Git hooks (Husky)
- VS Code settings

****Files/Modules:****

...

```
|— .eslintrc.js
|— .prettierrc.js
|— tsconfig.json
|— tsconfig.server.json
|— tsconfig.client.json
|— .husky/
|   |— pre-commit
|   |— pre-push
|— .vscode/
|   |— settings.json
|   |— extensions.json
|   |— launch.json
|— .editorconfig
```



FINAL SUMMARY

Total Component Count: 74 Major Components

Category Breakdown:

1. **Authentication & Authorization:** 3 components
2. **Organization Management:** 2 components
3. **Customer Management:** 2 components
4. **Product & Service Catalog:** 1 component
5. **AI Services:** 3 components
6. **Invoice Management:** 6 components
7. **Payment Management:** 2 components
8. **Expense Management:** 1 component
9. **Accounting & Bookkeeping:** 3 components
10. **Tax Calculation & Compliance:** 2 components

- 11. **WhatsApp Integration:** 3 components
- 12. **Communication Services:** 1 component
- 13. **Storage & File Management:** 1 component
- 14. **Notification System:** 1 component
- 15. **Background Jobs & Queue:** 1 component
- 16. **SAT Catalog Management:** 1 component
- 17. **Search & Filtering:** 1 component
- 18. **Analytics & Reporting:** 1 component
- 19. **Security & Compliance:** 3 components
- 20. **API & Integration:** 5 components
- 21. **Frontend Components:** 11 components
- 22. **Testing:** 3 components
- 23. **Deployment & Infrastructure:** 6 components
- 24. **Documentation:** 2 components
- 25. **Utilities:** 6 components
- 26. **Development Tools:** 2 components

This is the **complete technical inventory** of every component you need to build. Each component is fully detailed with its purpose, technical implementation, inputs/outputs, dependencies, and file structure.